

Chapter 3: Processes

Processes are the fundamental management and containment objects in Windows. Everything that executes must be under some process context, there is no such thing as running outside of a process. This chapter examines processes from multiple points of view - from creating, to managing, to destroying and almost everything in between.

In this chapter:

- **Process Basics**
 - **Process Creation**
 - **Creating Processes**
 - **Process Termination**
 - **Enumerating Processes**
-

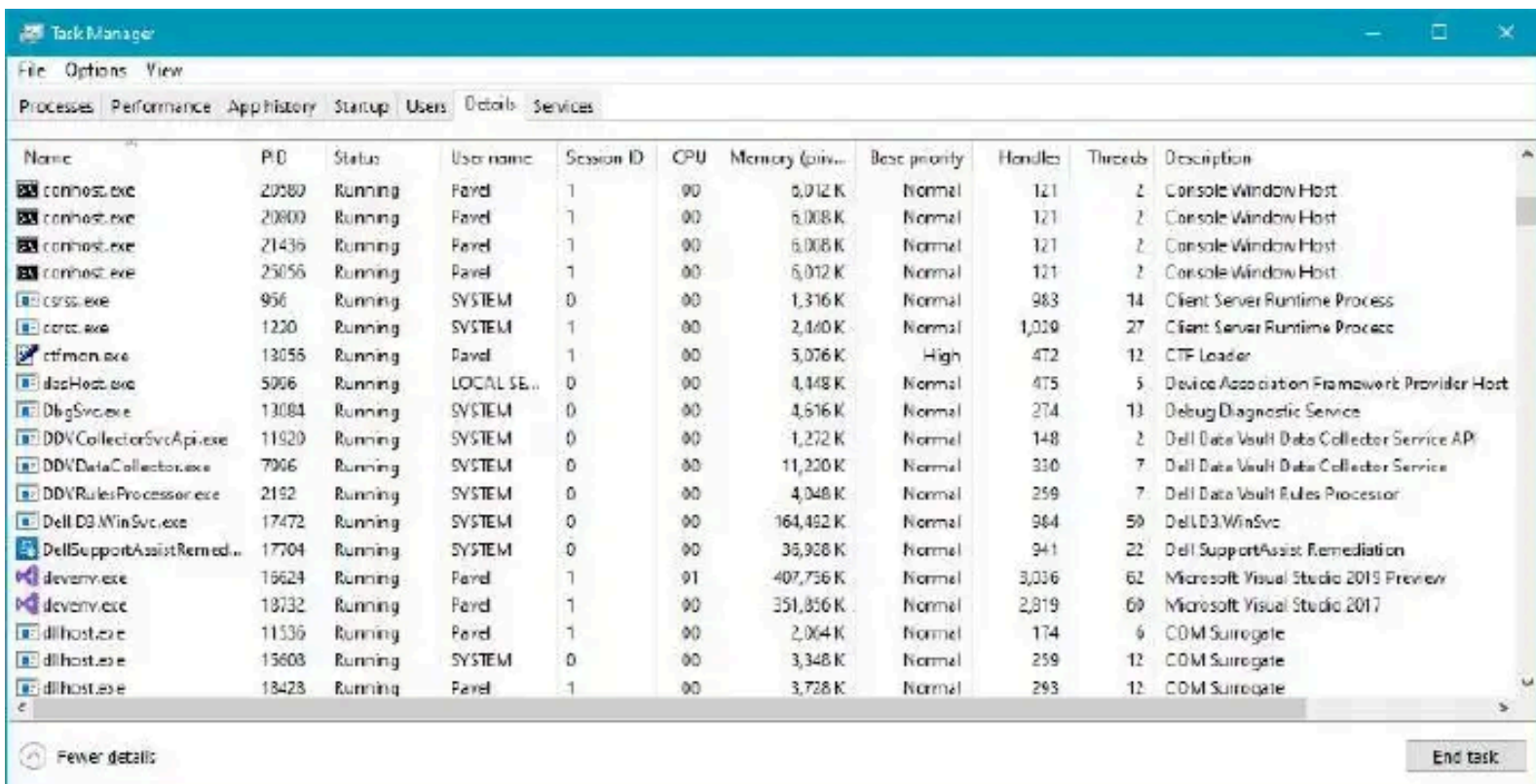
Process Basics

Although the basic structure and attributes of processes did not change since the first release of Windows NT, new process types have been introduced into the system with special behaviors or structure. The following is a quick overview of all process types currently supported, while later sections in this chapter discuss each process type in greater detail.

- **Protected Processes** - These processes were introduced in Windows Vista. They were created to support *Digital Rights Management* (DRM) protection by preventing intrusive access to processes rendering DRM-protected content. For example, no other process (even running with administrator rights) can read the memory within a protected process address space, so the DRM-protected data cannot be directly stolen.

- **UWP Processes** - These processes, available starting with Windows 8, host the Windows Runtime, and typically are published to the Microsoft Store. A UWP process executes inside an *AppContainer* - a sandbox of sorts that limits the operations this process can carry out.
- **Protected Processes Light (PPL)** - These processes (available from Windows 8.1) extended the protection mechanism from Vista by adding several levels of protection and even allowing third-party services to run as PPL, protecting them from intrusive access, and from termination, even by admin-level processes.
- **Minimal Processes** - These processes available from Windows 10 version 1607, is a truly new form of process. The address space of a minimal process does not contain the usual images and data structures that a normal process does. For example, there is no executable file mapped into the process address space, and no DLLs. The process address space is truly empty.
- **Pico Processes** - These processes are minimal processes with one addition: a Pico provider, which is a kernel driver that intercepts Linux system calls and translates them to equivalent Windows system calls. These processes are used with the *Windows Subsystem for Linux (WSL)*, available from Windows 10 version 1607.

A Process' basic information is easily visible in tools such as *Task Manager* and *Process Explorer*. Figure 3-1 shows *Task Manager Details* tab with some columns added beyond the defaults.



Name	PID	Status	User name	Session ID	CPU	Memory (priv...)	Base priority	Handles	Threads	Description
conhost.exe	20580	Running	Pavel	1	00	6,012 K	Normal	121	2	Console Window Host
conhost.exe	20800	Running	Pavel	1	00	6,008 K	Normal	121	2	Console Window Host
conhost.exe	21436	Running	Pavel	1	00	6,008 K	Normal	121	2	Console Window Host
conhost.exe	25056	Running	Pavel	1	00	6,012 K	Normal	121	2	Console Window Host
csrss.exe	956	Running	SYSTEM	0	00	1,316 K	Normal	983	14	Client Server Runtime Process
csrss.exe	1220	Running	SYSTEM	1	00	2,460 K	Normal	1,019	27	Client Server Runtime Process
ctfmon.exe	13056	Running	Pavel	1	00	5,076 K	High	472	12	CTF Loader
disHost.exe	5096	Running	LOCAL SE...	0	00	4,448 K	Normal	475	5	Device Association Framework Provider Host
DbgSvc.exe	13084	Running	SYSTEM	0	00	4,616 K	Normal	274	13	Debug Diagnostic Service
DDVCollectorSvcApi.exe	11920	Running	SYSTEM	0	00	1,272 K	Normal	148	2	Dell Data Vault Data Collector Service API
DDVDataCollector.exe	7966	Running	SYSTEM	0	00	11,220 K	Normal	330	7	Dell Data Vault Data Collector Service
DDVRulesProcessor.exe	2192	Running	SYSTEM	0	00	4,048 K	Normal	259	7	Dell Data Vault Rules Processor
Dell.D9.WinSvc.exe	17472	Running	SYSTEM	0	00	164,492 K	Normal	964	59	Dell.D9.WinSvc
DellSupportAssistRemedi...	17704	Running	SYSTEM	0	00	36,908 K	Normal	941	22	Dell SupportAssist Remediation
devany.exe	16624	Running	Pavel	1	91	407,736 K	Normal	3,036	62	Microsoft Visual Studio 2019 Preview
devany.exe	18732	Running	Pavel	1	00	351,856 K	Normal	2,819	69	Microsoft Visual Studio 2017
dllhost.exe	11536	Running	Pavel	1	00	2,064 K	Normal	174	6	COM Surrogate
dllhost.exe	15608	Running	SYSTEM	0	00	3,348 K	Normal	259	12	COM Surrogate
dllhost.exe	18428	Running	Pavel	1	00	3,728 K	Normal	293	12	COM Surrogate

Figure 3-1: Task Manager's Details tab

Let's briefly examine the columns appearing in figure 3-1:

Name

This is normally the executable name upon which the process is based on. Remember that this name is not the unique identifier of the process. Some processes don't seem to have an executable name at all. Examples include *System*, *Secure System*, *Registry*, *Memory Compression*, *System Idle Process* and *System Interrupts*.

- *System Interrupts* is not really a process, it's just used as a way to measure the time spent in the kernel servicing hardware interrupts and *Deferred Procedure Calls*. Both are beyond the scope of this book. You can find more information in the *Windows Internals* and *Windows Kernel Programming* books.
- *System Idle Process* is also not a real process. It always has a Process ID (PID) of zero. It accounts for Windows idle time. This is where the CPUs go when there is nothing to do.
- The *System* process is a true process, technically being a minimal process as well. It always has a PID of 4. It represents everything going on in kernel space - the memory used by the kernel and kernel drivers, the open handles, threads and so on.
- The *Secure System* process is only available on Windows 10 and Server 2016 (and later) systems that boot with *Virtualization Based Security* enabled. It represents everything going on in the secure kernel. Refer to the *Windows Internals* book for more information.
- The *Registry* process is a minimal process available from Windows 10 version 1803 (RS4) that is used as "working area" for managing the registry, rather than using the Paged Pool as was done in previous versions. For the purposes of this book, it's an implementation detail that does not affect the way the registry is accessed programmatically.
- The *Memory Compression* process is a minimal process available on Windows 10 version 1607 (but not on servers) and holds compressed memory in its address space. Memory compression is a feature added in Windows 10 to conserve physical memory (RAM), especially useful for devices with limited resources such as phones and IoT (*Internet of Things*) devices. Confusingly, *Task Manager* does not show this process, but it is shown properly by *Process Explorer*.



The reason *Memory Compression* is not shown in *Task Manager* is somewhat funny. In Windows 10 before version 1607 memory compression was supported, but the compressed memory was stored in the user-mode address space of the *System* process and this made the *System* process look as though it was consuming (possibly) a lot of memory. That was *compressed* memory, so it was really *saving* memory, but appearances are sometimes more important, and so the compressed memory was moved into its own (minimal) process and the process itself was hidden purposefully from the list in *Task Manager*.

In the rest of this chapter, up to the section “Minimal and Pico Processes”, deals with “normal” processes that are based on executable files. In any case, minimal and Pico processes can only be created by the kernel.

PID

The unique ID of the process. PIDs are multiple of 4, where the lowest valid PID value is 4 (belonging to the *System* processes). Process IDs are reused once a process terminates, so it’s possible to see a new process with a PID that was once used for a (now gone) process. If a unique identifier is required for a process, then a combination of the PID and the process start time is truly unique on a certain system.

You may recall from chapter 2, that handles also start with 4 and are multiples of 4, just like PIDs. This is not a coincidence. In fact, PIDs (and thread IDs) are handle values in a special handle table used just for this purpose.

Status

The *Status* column is a curious one. It can have one of three values: *Running*, *Suspended* and *Not Responding*. Let’s look at each one. Table 3-1 summarizes the meaning of these states based on process type.

Table 3-1: The Status column

Process type	Running when...	Suspended when...	Not responding when...
GUI process (not UWP)	GUI thread is responsive	All threads in the process are suspended	GUI thread does not check message queue for at least 5 seconds

Table 3-1: The Status column

Process type	Running when...	Suspended when...	Not responding when...
CUI process (not UWP)	At least one thread is not suspended	All threads in the process are suspended	Never
UWP process	in the foreground	in the background	GUI thread does not check message queue for at least 5 seconds

A process with a GUI must have at least one thread that handles its user interface. This thread has a message queue, created for it as soon as it called any UI or GDI function. This thread must therefore pump messages - that is, listen to its message queue and process messages that arrive. The typical listening functions are `GetMessage` or `PeekMessage`. If none is called for at least 5 seconds, *Task Manager* changes the status to *Not Responding*, the windows owned by that thread become faded and “(Not Responding)” is added to the window’s caption. The problematic thread did not examine its message queue for one of three possible reasons:

- It was suspended for whatever reason.
- It’s waiting for some I/O operation to complete, and it takes longer than 5 seconds.
- It’s doing some CPU intensive work that takes longer than 5 seconds.

We’ll look at these issues in chapter 5 (“Thread Basics”).

UWP processes are special in the sense that they are suspended unwillingly when they move into the background such as when the application’s window is minimized. A simple experiment can verify this case: Open the modern Calculator on Windows 10, and locate it in *Task Manager*. You should see its status as *Running*, meaning it can respond to user input and generally do its thing. Now minimize Calculator, and you’ll see the status changing after a few brief seconds to *Suspended*. This kind of behavior exists for UWP processes only.

Non-UWP processes that have no GUI are always shown with a *Running* status, as Windows has no idea what these processes are actually doing (or not). The only exception is if all threads in such a process are suspended, and then its status changes to *Suspended*.

The Windows API does not have a function to suspend a process, only to suspend a thread. Technically it’s possible to loop over all the threads in a certain process and suspend each one (assuming a powerful enough handle can be obtained). The native API (implemented in `NtDll.dll`) does have a function for that purpose, `NtSuspendProcess`. This is the function called by *Process Explorer* if you right-click a process and choose *Suspend*. Of course the

opposite function exists as well - *NtResumeProcess*.

User Name

The user name indicates under what user the process is running. A token object is attached to the process (called *Primary Token*) that holds the security context for the process based on the user. That security context contains information such as the groups the user belongs to, the privileges it has and more. We'll take a deeper look at tokens in chapter 16. A process can run under special built-in users, such as Local System (shown as *System* in Task Manager), Network Service and Local Service. These user accounts are typically used to run services, which we'll look at in chapter 16.

Session ID

The session number under which session the process executes. Session 0 is used for system process and services and session 1 and higher are used for interactive logins. We'll look at sessions in more detail in chapter 16.

CPU

This column shows the CPU percentage consumption for that process. Note it shows whole numbers only. To get better accuracy, use *Process Explorer*.

Memory

The memory-related columns are somewhat tricky. The default column shown by *Task Manager* is *Memory (active private working set)* (Windows 10 version 1903) or *Memory (private working set)* (earlier versions). The term *Working Set* means RAM (physical memory). *Private working set* is the RAM used by the process and not shared with other processes. The most common example of shared memory is for DLL code. *Active private working Set* is the same as *Private working set*, but is set to zero for UWP processes that are currently suspended.

Are the above two counters a good indication of the amount of memory used by a process? Unfortunately, no. These indicate the private RAM used, but what about memory that is currently paged out? There is another column for that - *Commit Size*. This is the best column to use to get a sense of the memory usage for a process. The "unfortunate" part is that *Task Manager* does not show this column by default.



Process Explorer has an equivalent column for *Commit Size*, but it's called *Private Bytes*, which is consistent with a *Performance Counter* by that name.

These memory terms are discussed further in chapter 12.

Base Priority

The *Base Priority* column, officially called *Priority Class*, shows one of six values, that provide the base scheduling priority for threads executing in that process. The possible values with the priority level associated with them are the following:

- *Idle* (called *Low* in *Task Manager*) = 4
- *Below Normal* = 6
- *Normal* = 8
- *Above Normal* = 10
- *High* = 13
- *Real-time* = 24

The most common (and default) priority class is *Normal* (8). We'll discuss priorities and scheduling in chapter 6.

Handles

The *handles* column shows the number of handles to kernel objects that are open in a particular process. This was discussed at length in chapter 2.

Threads

The *Threads* column shows the number of threads in each process. Normally, this should be at least one, as a process with no threads is useless. However, some processes are shown with no threads (using a dash). Specifically, *Secure System* is shown with no threads because the secure kernel actually uses the normal kernel for scheduling. The *system Interrupts* pseudo-process is not a process at all, so cannot have any threads. Lastly, the *System Idle Process* does not own threads either. The number of threads shown for this process is the number of logical processors on the system.

There are other columns of interest in *Task Manager*, which will be examined in due course.

Processes in Process Explorer

Process Explorer can be thought of as a “Task Manager on steroids”. It has most of the functionality of *Task Manager* and much more. We’ve already seen its ability to show open handles in processes. In this section we’ll examine some of its process-related capabilities.

First, *Process Explorer* can show processes with various columns, just like *Task Manager*. It has more columns than are available in *Task Manager*, however. Immediately apparent are the colors processes are shown in. Each color indicates some interesting aspect of a process. Of course a process can have more than one such “aspect”, worthy of a color, and in that case one color “wins” and the “losing” color is not shown. All the available colors can be changed and enabled or disabled by selecting *Options, Configure Colors...* from the menu, showing the dialog in figure 3-2.

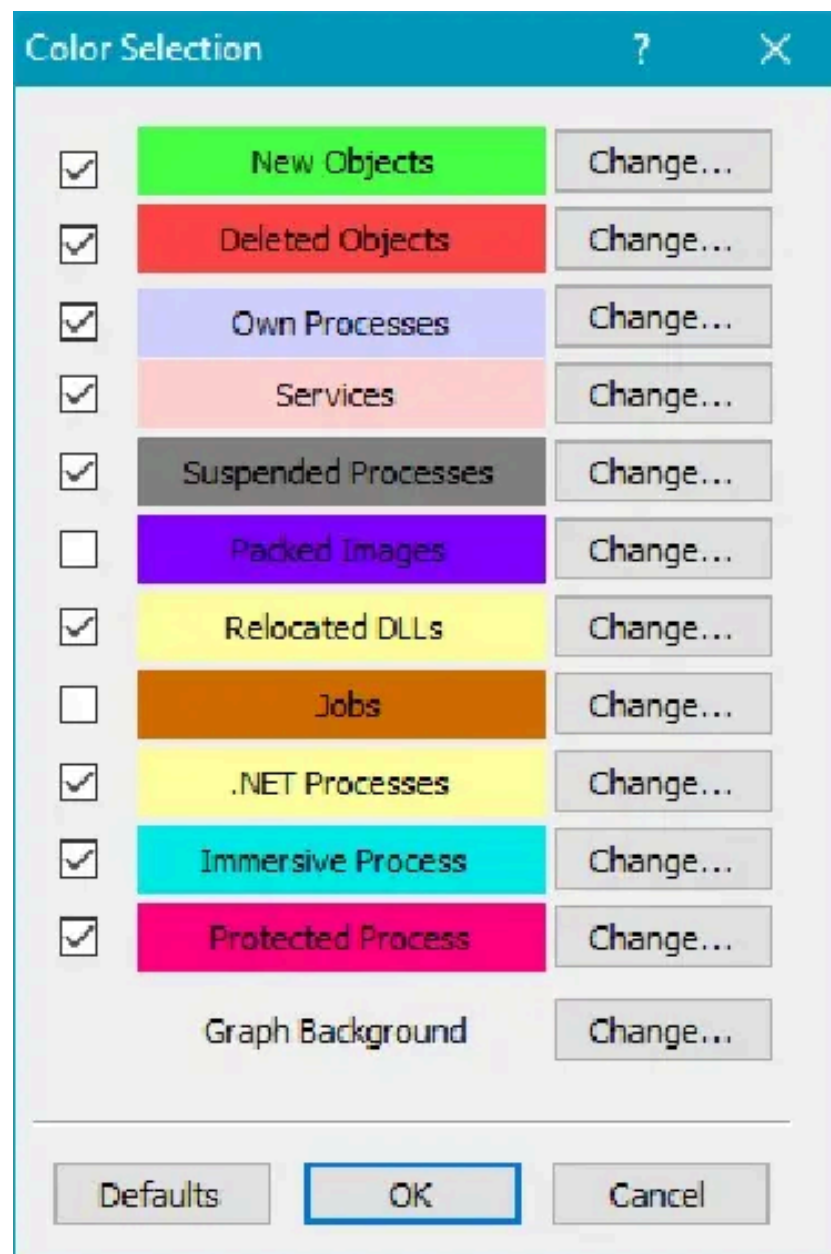


Figure 3-2: Color configurations in Process Explorer

Table 3-2 summarizes their background colors and meaning.

Table 3-2: Colors in Process Explorer

Name (default color)	Meaning
New Objects (green)	New objects created
Deleted Objects (red)	Objects that have been destroyed
Own Processes (blueish)	Processes running under the logged on user account
Services (pink)	Processes hosting Windows Services (see chapter 19)
Suspended Processes (gray)	Suspended processes
Packed Images (purple)	Executables or DLLs that use packing techniques to reduce size. In some cases, malware may be using such techniques
Relocated DLLs (yellowish)	Shown in the modules view (not in the main process view). Discussed in chapter 15
Jobs (brown)	Processes that are part of a job (see chapter 4)
.NET Processes (yellowish)	Processes that run some .NET code. More precisely, processes that host the .NET CLR
Immersive Processes (cyan)	Normally UWP processes (that are not suspended). More precisely, processes that host the Windows Runtime. The function used to determine this is <code>IsImmersiveProcess</code>
Protected Processes (fuchsia)	Protected processes and PPL processes (see later in this chapter)
(all other) (white)	Processes that don't have any of the enabled aspects. If all colors are enabled, what's left are mostly system processes

I personally added the protected processes color and selected the default to be Fuchsia (unrelated to Google's new OS).

The new and destroyed objects colors are shown for a period of one second by default. You can make it longer by opening the menu *Options, Difference Highlight Duration....*

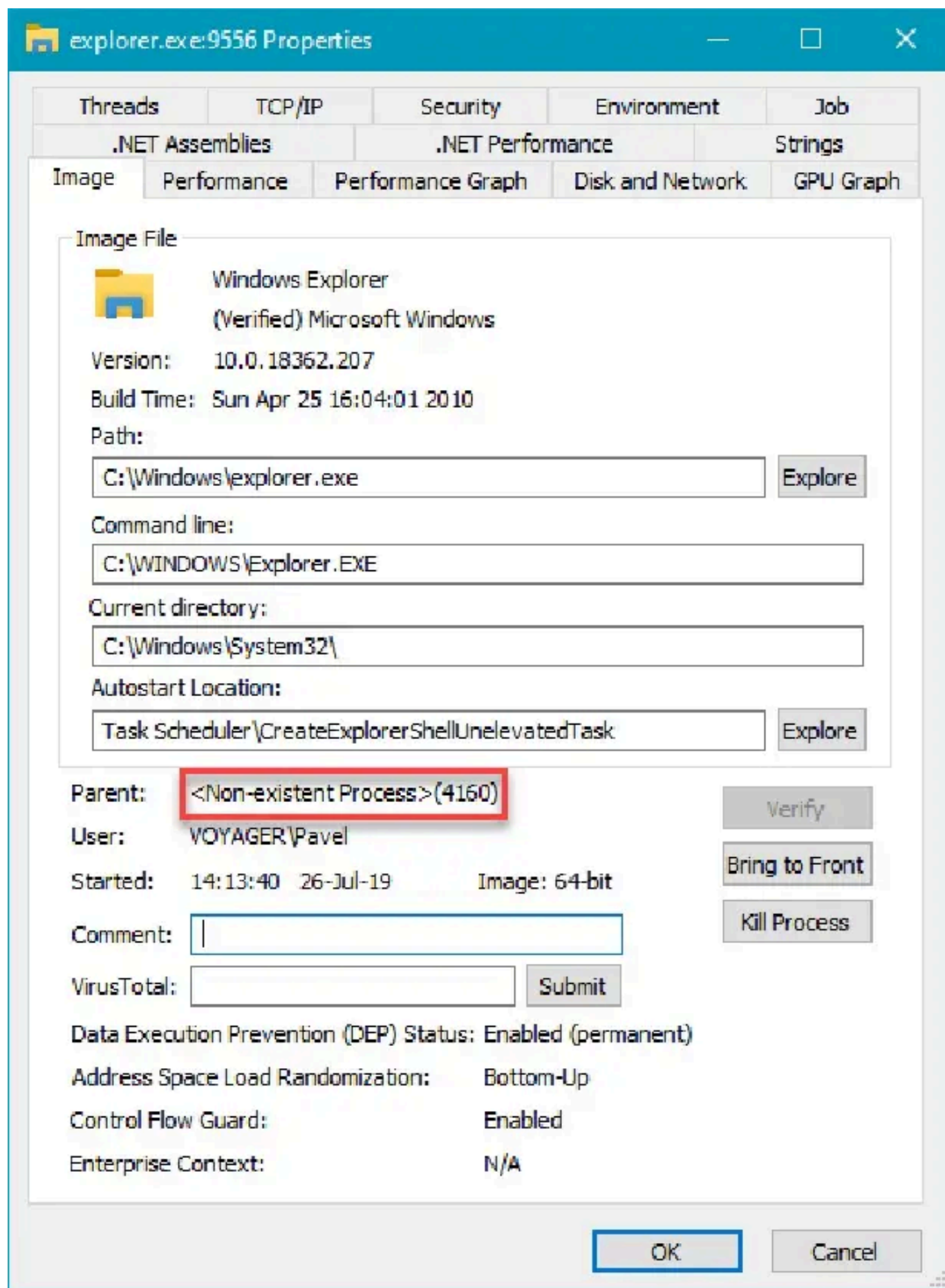
Another interesting feature of *Process Explorer* is the ability to “sort” processes in a tree (more accurately, trees) of processes. If you click on the *Process* column where the image name is, you can sort normally, but a third click turns the *Process* column into trees of processes. Part of these trees is shown in figure 3-3.

Process	PID	CPU	Private Bytes	Working Set	Description
MsMpEng.exe	21552	0.14	226,652 K	155,420 K	Antimalware Service Executable
NisSrv.exe	21488		11,060 K	10,696 K	Microsoft Network Realtime Insp...
svchost.exe	17760		1,740 K	3,612 K	Host Process for Windows Servi...
svchost.exe	25996		2,400 K	3,816 K	Host Process for Windows Servi...
svchost.exe	20224		2,316 K	3,888 K	Host Process for Windows Servi...
UploaderService.exe	3056		2,372 K	4,556 K	TechSmith Uploader Service
svchost.exe	15112		2,564 K	4,720 K	Host Process for Windows Servi...
svchost.exe	10880		9,712 K	19,772 K	Host Process for Windows Servi...
svchost.exe	28532		3,776 K	8,452 K	Host Process for Windows Servi...
Lsass.exe	1300		968 K	2,380 K	
lsass.exe	1316	< 0.01	14,360 K	18,580 K	Local Security Authority Process
fontdrvhost.exe	1464		1,828 K	2,072 K	Usermode Font Driver Host
csrss.exe	1220	0.05	4,408 K	3,728 K	Client Server Runtime Process
winlogon.exe	1792		2,824 K	6,248 K	Windows Logon Application
fontdrvhost.exe	1860		9,748 K	15,576 K	Usermode Font Driver Host
dwm.exe	1944	0.61	193,404 K	113,436 K	Desktop Window Manager
explorer.exe	9556	0.08	174,200 K	113,692 K	Windows Explorer
TSVNCache.exe	11952	< 0.01	3,780 K	4,668 K	TortoiseSVN status cache
SecurityHealthSystray.exe	8688		1,880 K	3,532 K	Windows Security notification icon
RtkAudUService64.exe	14792		3,156 K	6,188 K	Realtek HD Audio Universal Ser...
WavesSvc64.exe	14472		30,648 K	9,076 K	Waves MaxxAudio Service Appli...
OneDrive.exe	4208		238,520 K	29,440 K	Microsoft OneDrive
jetbrains-toolbox.exe	17084		51,176 K	42,620 K	JetBrains Toolbox
jetbrains-toolbox-helper.exe	17348		48,460 K	55,156 K	
ONENOTEM.EXE	17596		2,368 K	1,252 K	Send to OneNote Tool
devenv.exe	18732	0.91	578,128 K	219,252 K	Microsoft Visual Studio 2017
PerfWatson2.exe	18740	< 0.01	45,680 K	18,788 K	PerfWatson2.exe
ServiceHub.Host.Node.x86.exe	20428		19,348 K	1,828 K	Node.js: Server-side JavaScript
conhost.exe	20448	< 0.01	6,540 K	3,052 K	Console Window Host
ServiceHub.VSDetouredH...	6096		115,928 K	8,652 K	ServiceHub.Host.CLR.x86
ServiceHub.IdentityHost.exe	19828		29,628 K	18,156 K	ServiceHub.Host.CLR.x86
ServiceHub.Host.CLR.x86...	6112	0.01	54,248 K	19,332 K	ServiceHub.Host.CLR.x86
ServiceHub.SettingsHost.e...	6896		67,156 K	20,796 K	ServiceHub.Host.CLR.x86
vsvnvswrk.exe	20324		1,808 K	1,256 K	VisualSVN host process for Visua...
vcpgksrv.exe	21936	< 0.01	3,984 K	3,220 K	Microsoft (R) Visual C++ Packag...
vcpgksrv.exe	13012	< 0.01	5,076 K	3,308 K	Microsoft (R) Visual C++ Packag...
vcpgksrv.exe	27136	< 0.01	4,996 K	8,216 K	Microsoft (R) Visual C++ Packag...
Snagit32.exe	5984	0.30	186,288 K	107,064 K	Snagit
SnagitPriv.exe	21920		1,928 K	5,624 K	Snagit RPC Helper
SnagitEditor.exe	15248	< 0.01	286,700 K	227,952 K	Snagit Editor
OUTLOOK.EXE	12152		277,548 K	178,548 K	Microsoft Outlook
Code.exe					

CPU Usage: 7.46% Commit Charge: 67.07% Processes: 330 Physical Usage: 45.24%

Figure 3-3: Process tree(s) in Process Explorer

Each child node in the tree is a child processes of its parent node. Some processes seem to be left-justified (see *Explorer.exe* in figure 3-3). These processes don't have a parent process, or more accurately - had a parent process that has since exited. Double-clicking such a process and switching to the *Image* tab shows basic information about the process including its parent. Figure 3-4 shows this information for that instance of *Explorer.exe*.

Figure 3-4: *Explorer.exe* Properties

Notice the parent process is unknown but its PID is known (4160 in figure 3-4). This means that the parent PID is stored with the child process, but if the parent no longer exists no

other information about it remains.

You may be wondering what would happen if a new process is created with PID 4160 in figure 3-4, since PIDs get reused. Fortunately, *Process Explorer* is not confused, as it checks the start time of the parent process. If it's later than the child then clearly that process cannot be the parent.



Why is *Explorer.exe* parentless? This is actually the normal case, as Explorer is created by an earlier process running *UserInit.exe*, whose job (among other things) is to launch the default shell (configured in the registry by default to be *Explorer.exe*). Once its work is done, the *UserInit* process simply exits.

The important point to remember about this parent-child process relationship is this: if process A creates process B and process A dies, process B is unaffected. In other words, processes in Windows are more like siblings - they don't affect each other after creation.

Process Creation

The major parts involved in process creation is depicted in figure 3-5.

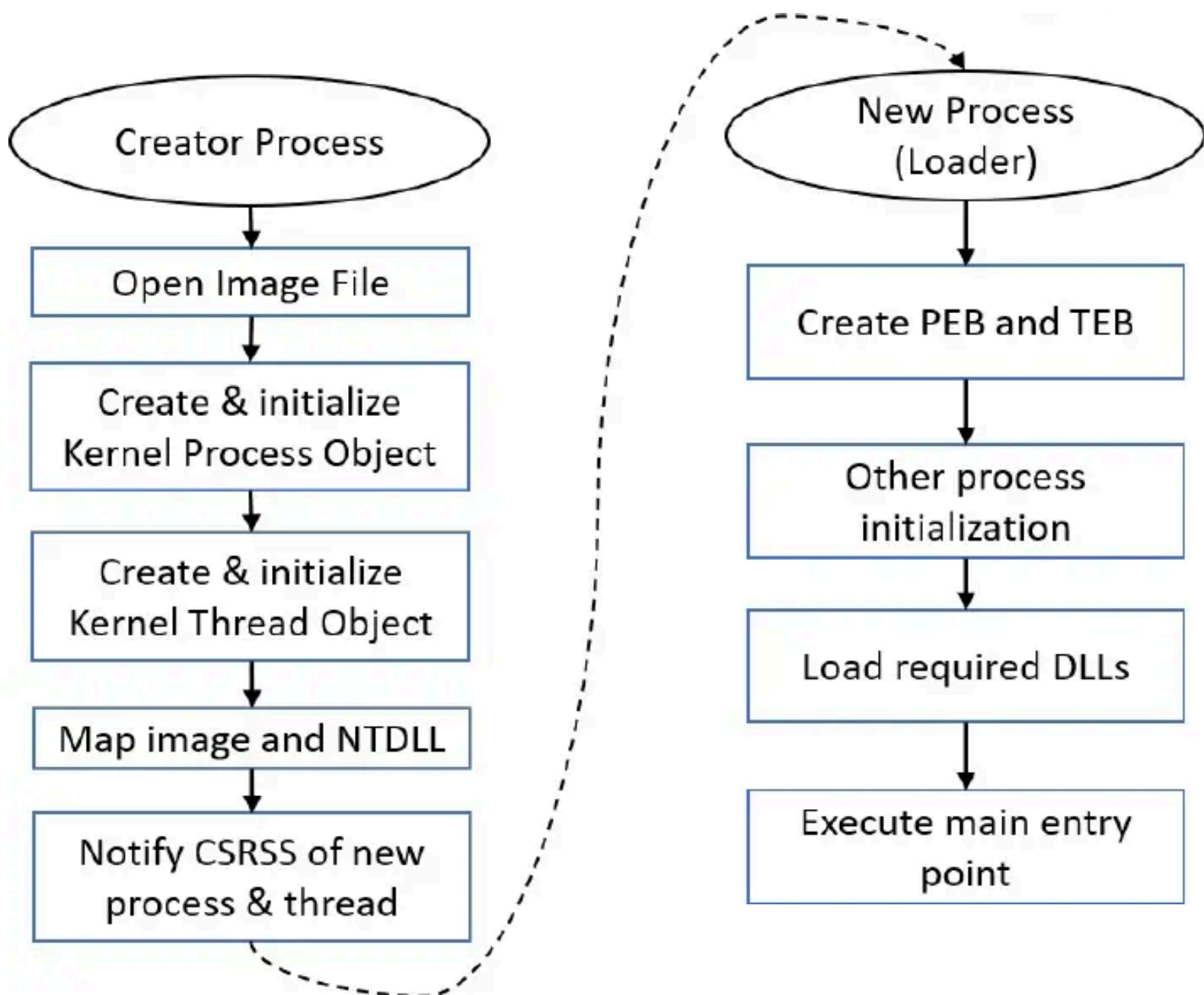


Figure 3-5: Flow of process creation

First, the kernel opens the image (executable) file and verifies that it's in the proper format known as *Portable Executable* (PE). The file extension does not matter, by the way - only the actual content does. Assuming the various headers are valid, the kernel then creates a new process kernel object and a thread kernel object, because a normal process is created with one thread that eventually should execute the main entry point.

At this point, the kernel maps the image to the address space of the new process, as well as *NtDll.Dll*. *NtDll* is mapped to every process (except Minimal and Pico processes), since it has very important duties in the final stage of process creation (outlined below) as well being the trampoline from which system calls are invoked. The final major step which is still carried out by the creator process is notifying the Windows subsystem process (*Csrss.exe*) of the fact that a new process and thread have been created. (*Csrss* can be thought of as a helper to the kernel for managing some aspects of Windows subsystem processes).

At this point, from the kernel's point of view, the process has been created successfully, so the process creation function invoked by the caller (typically `CreateProcess`, discussed in the next section) returns success. However, the new process is not yet ready to execute its initial code. The second part of process initialization must be carried out in the context of the new process, by the newly created thread.

Some developers believe that the first thing that runs in a new process is the executable's main function. This is far from the truth, however. There is a lot going on before the actual main function starts running. The star of this part is *NtDll*, as there is no other OS level code in the process at this time. *NtDll* has several duties at this point.

First, it creates the user-mode management object for the process, known as the *Process Environment Block* (PEB) and the user mode management object for the first thread called *Thread Environment Block* (TEB). These structures are partially-documented (in `<winternl.h>`), and officially should not be used directly by developers. That said, there are cases where these structures are useful, especially when trying to achieve things that are difficult to do otherwise.

The current thread's TEB is accessible via `NtCurrentTeb()`, while the PEB of the current process is available via `NtCurrentTeb()->ProcessEnvironmentBlock`.

Then some other initializations are carried out, including the creation of the default process heap (see chapter 13), creation and initialization of the default process thread pool (chapter 9) and more. For full details, consult the *Windows Internals* book.

The last major part before the entry point can start execution is to load required DLLs. This part of *NtDll* is often referred to as the *Loader*. The loader looks at the import section of the executable, which includes all the libraries the executable depends upon. These typically include the Windows subsystem DLLs such as *kernel32.dll*, *user32.dll*, *gdi32.dll* and *advapi32.dll*.

To get a sense for these import libraries, we can use the *DumpBin.exe* tool available as part of the Windows SDK and Visual Studio installation. Open the Developer Command Prompt to gain easy access to the various tools and type the following to look at the imports for *Notepad.exe*:

```
c:\>dumpbin /imports c:\Windows\System32\notepad.exe
```

The result is a dump of all import libraries and what symbols are imported (used) from those libraries. Here is an abbreviated output (Windows 10 version 1903):

Dump of file c:\Windows\System32\notepad.exe

File Type: EXECUTABLE IMAGE

Section contains the following imports:

```
GDI32.dll
    140022788 Import Address Table
    1400289E8 Import Name Table
        0 time date stamp
        0 Index of first forwarder reference
        35C SelectObject
        2D0 GetTextFaceW
        1C2 EnumFontsW
...
USER32.dll
    140022840 Import Address Table
    140028AA0 Import Name Table
        0 time date stamp
        0 Index of first forwarder reference
        364 SetThreadDpiAwarenessContext
        2AD PostMessageW
        BA DialogBoxParamW
...
msvcrt.dll
    140022FD8 Import Address Table
    140029238 Import Name Table
        0 time date stamp
        0 Index of first forwarder reference
        2F ?terminate@@YAXXZ
        496 memset
...
api-ms-win-core-libraryloader-l1-2-0.dll
    140022C60 Import Address Table
    140028EC0 Import Name Table
        0 time date stamp
```

```

        0 Index of first forwarder reference
          F GetModuleFileNameA
        18 LoadLibraryExW
        13 GetModuleHandleExW
...
urlmon.dll
        00000001 Characteristics
        000000014002C0D0 Address of HMODULE
        000000014002F0E0 Import Address Table
        0000000140028368 Import Name Table
        0000000140028638 Bound Import Name Table
        0000000000000000 Unload Import Name Table
          0 time date stamp
                                0000000140020F31    3B FindMimeFromData
...

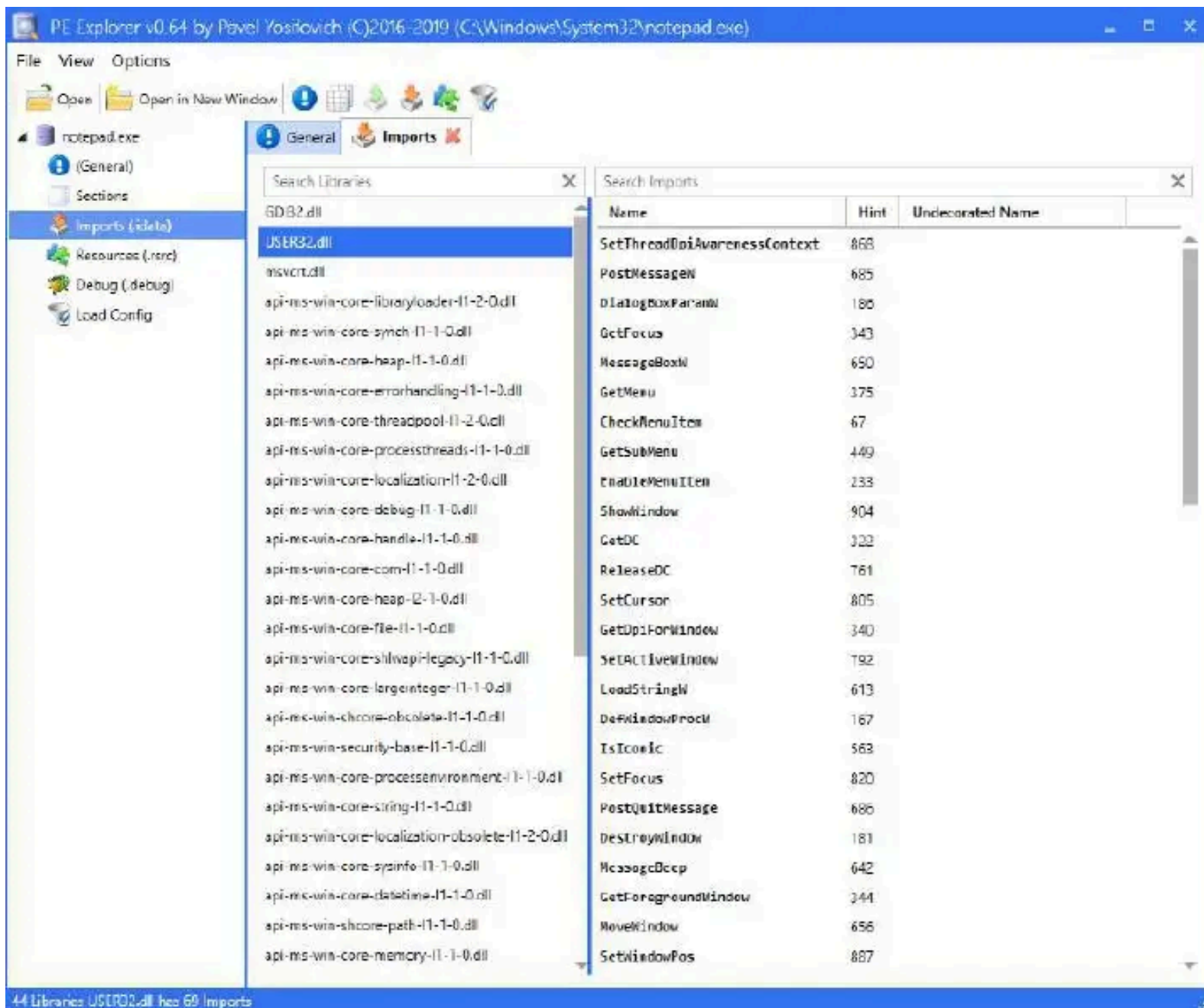
```

For each required DLL, `dumpbin` shows the imported functions from that DLL, i.e. the functions actually used by the executable. Some of the DLL names may look weird, and in fact you won't find them as actual files. The example in the above output is *api-ms-win-core-libraryloader-l1-2-0.dll*. This is known as an *API Set*, which is an indirect mapping from a contract (the API Set) to an actual implementation DLL (sometimes referred to as a *Host*).



API Sets exist starting from Windows 7.

Another way to view these dependencies is with a graphical. Figure 3-6 shows one such tool, *PE Explorer*, downloadable from <http://github.com/zodiacon/AllTools>, with *Notepad.exe*'s dependencies. For each API Set or DLL, it shows the imported functions.

Figure 3-6: PE Explorer with *Notepad.exe*

API Sets allows Microsoft to separate function “declarations” from the actual implementation. This means the implementing DLL can change at a later Windows release, and can even be different on different form factors (IoT devices, HoloLens, Xbox, etc.). The actual mapping between API set and implementation is stored for each process in the PEB. You can view these mappings using the *ApiSetMap.exe* tool, downloadable from <https://github.com/zodiacon/WindowsInternals/releases>. Here is the first few lines of output:

C:\>APISetMap.exe

ApiSetMap - list API Set mappings - version 1.0

(c) Alex Ionescu, Pavel Yosifovich, and Contributors

<http://www.alex-ionescu.com>

```

    api-ms-oncoreuap-print-render-l1-1-0.dll -> s{printrenderapihost.dll}
    api-ms-win-appmodel-identity-l1-2-0.dll -> s{kernel.appcore.dll}
api-ms-win-appmodel-runtime-internal-l1-1-6.dll -> s{kernel.appcore.dll}
    api-ms-win-appmodel-runtime-l1-1-3.dll -> s{kernel.appcore.dll}
    api-ms-win-appmodel-state-l1-1-2.dll -> s{kernel.appcore.dll}
    api-ms-win-appmodel-state-l1-2-0.dll -> s{kernel.appcore.dll}
    api-ms-win-appmodel-unlock-l1-1-0.dll -> s{kernel.appcore.dll}
    api-ms-win-base-bootconfig-l1-1-0.dll -> s{advapi32.dll}
        api-ms-win-base-util-l1-1-0.dll -> s{advapi32.dll}
    api-ms-win-composition-redirection-l1-1-0.dll -> s{dwmredir.dll}
api-ms-win-composition-windowmanager-l1-1-0.dll -> s{udwm.dll}
    api-ms-win-containers-cmclient-l1-1-1.dll -> s{cmclient.dll}
        api-ms-win-core-apiquery-l1-1-1.dll -> s{ntdll.dll}
        api-ms-win-core-apiquery-l2-1-0.dll -> s{kernelbase.dll}

```

The DLLs or API Set names don't have a full path associated with them. The Loader searches in the following directories in order until the DLL is located:

1. If the DLL name is one of the *KnownDLLs* (specified in the registry), the system directory is searched first (see item 4) (Known DLLs are described in chapter 15 in part 2). This is where the Windows subsystem DLLs reside (*kernel32.dll*, *user32.dll*, *advapi32.dll*, etc.)
2. The directory of the executable
3. The current directory of the process (determined by the parent process). (This is discussed in the next section)
4. The System directory returned by `GetSystemDirectory` (e.g. *c:\windows\system32*)
5. The Windows directory returned by `GetWindowsDirectory` (e.g. *c:\Windows*)
6. The directories listed in the *PATH* environment variable



The DLLs listed in the Known DLLs registry key (*HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\KnownDLLs*) are always loaded from the system directory, to prevent DLL hijacking, where an alternate DLL with the same name is placed in the executable's directory.

Once the DLL is found, it's loaded and its `DllMain` function (if exists) is called with the reason `DLL_PROCESS_ATTACH` indicating the DLL has now been loaded into a process. (Full discussion of DLL loading is saved for chapter 15).

This process continues recursively, because one DLL may depend on another DLL and so on. If any of the DLLs is not found, the Loader displays a message box similar to figure 3-7. Then the Loader terminates the process.



Figure 3-7: Failure to locate a required DLL

If any of the DLL's `DllMain` function returns `FALSE`, this indicates the DLL was not able to initialize successfully. Then the Loader halts further progress and shows the message box in figure 3-8, after which the process shuts down.

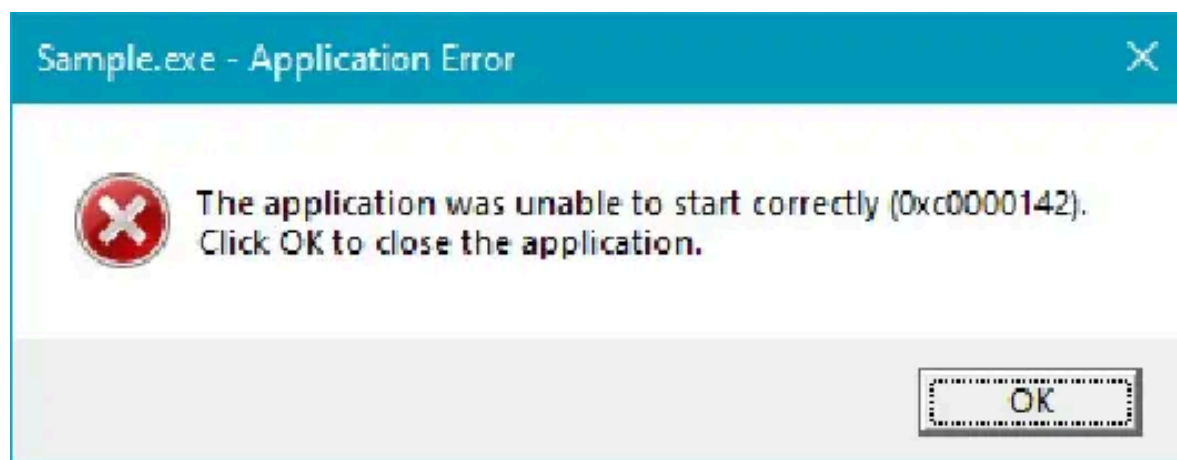


Figure 3-8: Failure to initialize a required DLL

Once all required DLLs have been loaded and initialized successfully, control transfers to the main entry point of the executable. The entry point in question is not the actual main function provided by the developer. Instead, it's a function provided by the C/C++ runtime, set appropriately by the linker. Why is that needed? Calling functions from the C/C++ runtime such as `malloc`, `operator new`, `fopen` and others require some setup. Also, global C++ objects must have their constructors called, even before your main function executes. All this is done by the C/C++ runtime startup function.

There are actually four main functions developers can write, and for each there is a corresponding C/C++ runtime function. Table 3-4 summarizes these names and when they are used.

Table 3-4: main and C/C++ startup functions

Developer's main	C/C++ runtime startup	Scenario
main	mainCRTStartup	Console application using ASCII characters
wmain	wmainCRTStartup	Console application using Unicode characters
WinMain	WinMainCRTStartup	GUI application using ASCII characters
wWinMain	wWinMainCRTStartup	GUI application using Unicode characters

The correct function is set by the linker's /SUBSYSTEM switch, also exposed through Visual Studio, in the Project Properties dialog shown in figure 3-9.

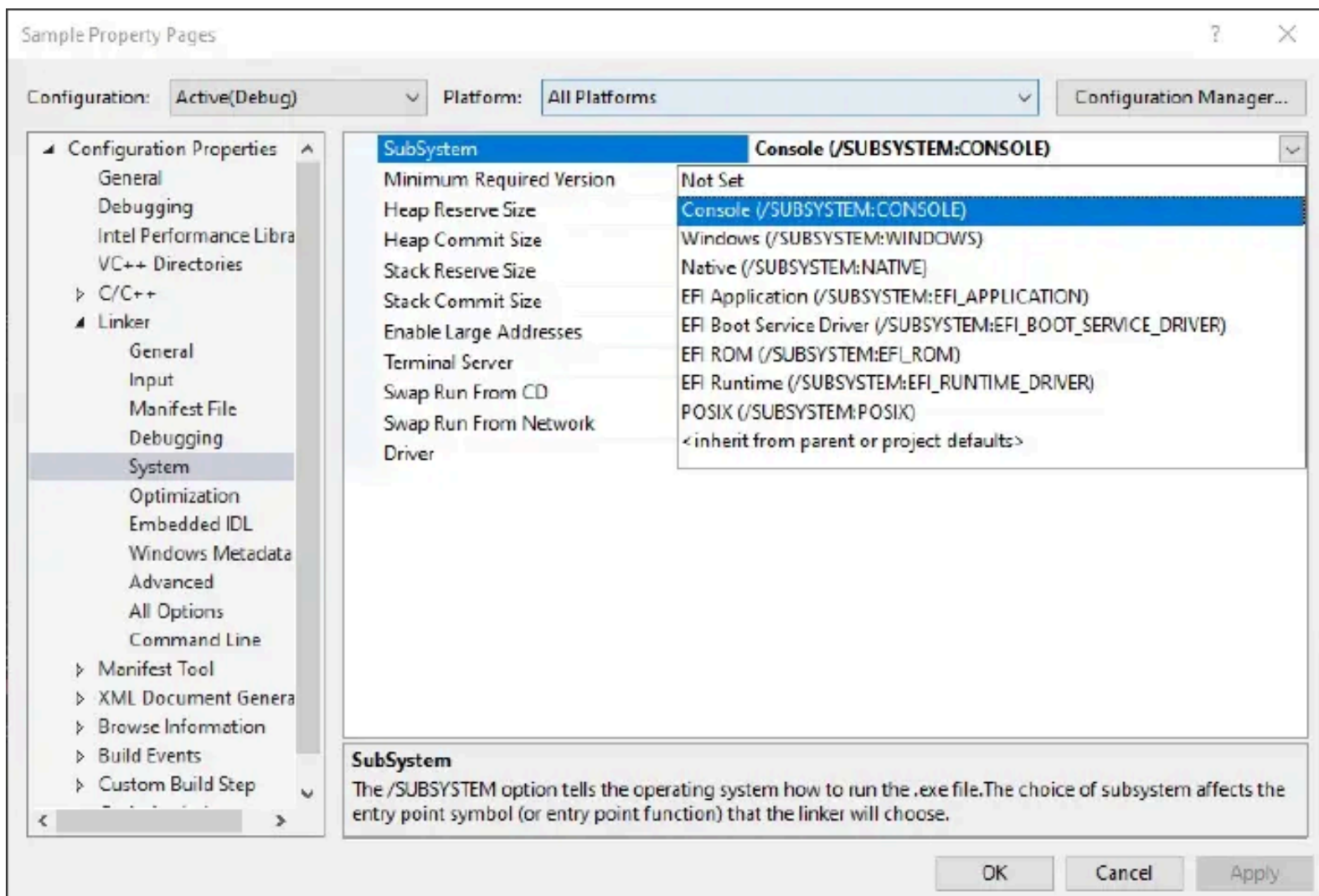


Figure 3-9: System Linker setting in Visual Studio

Is a console-based process that different from a GUI-based process? Not really. Both these types are members of the Windows subsystem. A console application can show GUI and a GUI application can use a console. The difference lies in various defaults such as the main

function prototype and whether a console window should be created by default.

A GUI application can create a console with `AllocConsole`.

The main Functions

Based on the rows in table 3-4, there are four variants of a main function written by developers:

```
int main(int argc, const char* argv[]);    // const is optional
int wmain(int argc, const wchar_t* argv[]); // const is optional
int WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
            LPSTR commandLine, int showCmd);
int wWinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
            LPWSTR commandLine, int showCmd);
```

Sometimes you'll see main functions written as `_tmain` or `_tWinMain`. As you probably guess, this allows compiling as Unicode or ASCII based on the compilation constants `_UNICODE` and `UNICODE`, respectively.

With the classic `main/wmain` functions, the command line arguments are broken down by the C/C++ runtime prior to calling `(w)main`. `argc` indicates the number of command-line arguments and is at least one, as the first “argument” is the full path of the executable. `argv` is an array of pointers to the parsed (split based on whitespace) arguments. This means `argv[0]` points to the full executable path.

With the `w(WinMain)` functions, the parameters are as follows:

- `hInstance` represents the executable module itself within the process address space. Technically, it's the address to which the executable is mapped. It's up to the linker to specify this value. By default, Visual Studio uses the linker option `/DYNAMICBASE` which generates a pseudo-random base address each time the project is built. In any

case, the number itself is not important, but it is needed in various functions, such as for loading resources (`LoadIcon`, `LoadString` and others).

The `HINSTANCE` type is just a `void` pointer. Incidentally, the `HMODULE` type which is sometimes used interchangeably with `HINSTANCE`, is indeed the same thing. The reason two types exist instead of one has to do with 16 bit Windows, where they meant different things.

- `hPrevInstance` should represent the `HINSTANCE` of a previous instance of the same executable. This value is always `NULL`, however, and is not really used. In the 16-bit Windows days, it was non-`NULL`. This means there is no direct way to know whether another process running the same executable already exists. We saw one way to deal with that if needed in the *Singleton* demo application from chapter 2. The `WinMain` signature was preserved from 16-bit Windows for easier porting to 32-bit Windows (at the time). The end result is that this parameter is often written without a variable name because it's completely useless.

An alternative technique to silence compiler warnings (especially with pure C) is to use the `UNREFERENCED_PARAMETER` macro with the variable name like so: `UNREFERENCED_PARAMETER(hPrevInstance);`. Ironically, this macro actually *references* its argument by simply writing it with a terminating semicolon; that's enough to make the compiler happy.

- `commandLine` is the command line string excluding the executable path - it's the rest of the command line (if any). It's not "parsed" into separate tokens - it's just a single string. If parsing is beneficial, the following function can be used:

```
#include <ShellApi.h>
```

```
LPWSTR* CommandLineToArgvW(_In_ LPCWSTR lpCmdLine, _Out_ int* pNumArgs);
```

The function takes the command line and splits it into tokens, returning a pointer to an array of string pointers. The count of strings is returned via `*pNumArgs`. The function allocates a block of memory to hold the parsed command-line arguments, and it must be eventually freed by calling `LocalFree`. The following code snippet shows how to parse the command line properly in a `wWinMain` function:

```
int wWinMain(HINSTANCE hInstance, HINSTANCE, LPWSTR lpCmdLine, int nCmdShow) {
    int count;
    PWSTR* args = CommandLineToArgvW(lpCmdLine, &count);

    WCHAR text[1024] = { 0 };
    for (int i = 0; i < count; i++) {
        ::wcscat_s(text, 1024, args[i]);
        ::wcscat_s(text, 1024, L"\n");
    }
    ::LocalFree(args);

    ::MessageBox(nullptr, text, L"Command Line Arguments", MB_OK);
}
```

If the string passed to `CommandLineToArgvW` is the empty string, then its return value is a single string which is the full executable path. On the other hand, if the passed-in string contains non-empty arguments, it returns an array of string pointers containing only the parsed arguments without having the full executable path as the first parsed string.

A process can get its command line at any time by calling `GetCommandLine`, and it's a proper argument to `CommandLineToArgvW`. This could be useful if parsing is required outside of `wWinMain`.

- `showCmd` is the last argument which suggests how to show the main window of the application. It's determined by the process creator, where the default is `SW_SHOWDEFAULT` (10). The application of course is free to disregard this value and show its main window in any way it pleases.

Process Environment Variables

Environment variables is a set of name/value pairs that can be set on a system or user wide basis using the dialog shown in figure 3-10 (accessible from the *System Properties* dialog or simply by searching). The names and values are stored in the Registry (like most system data in Windows).

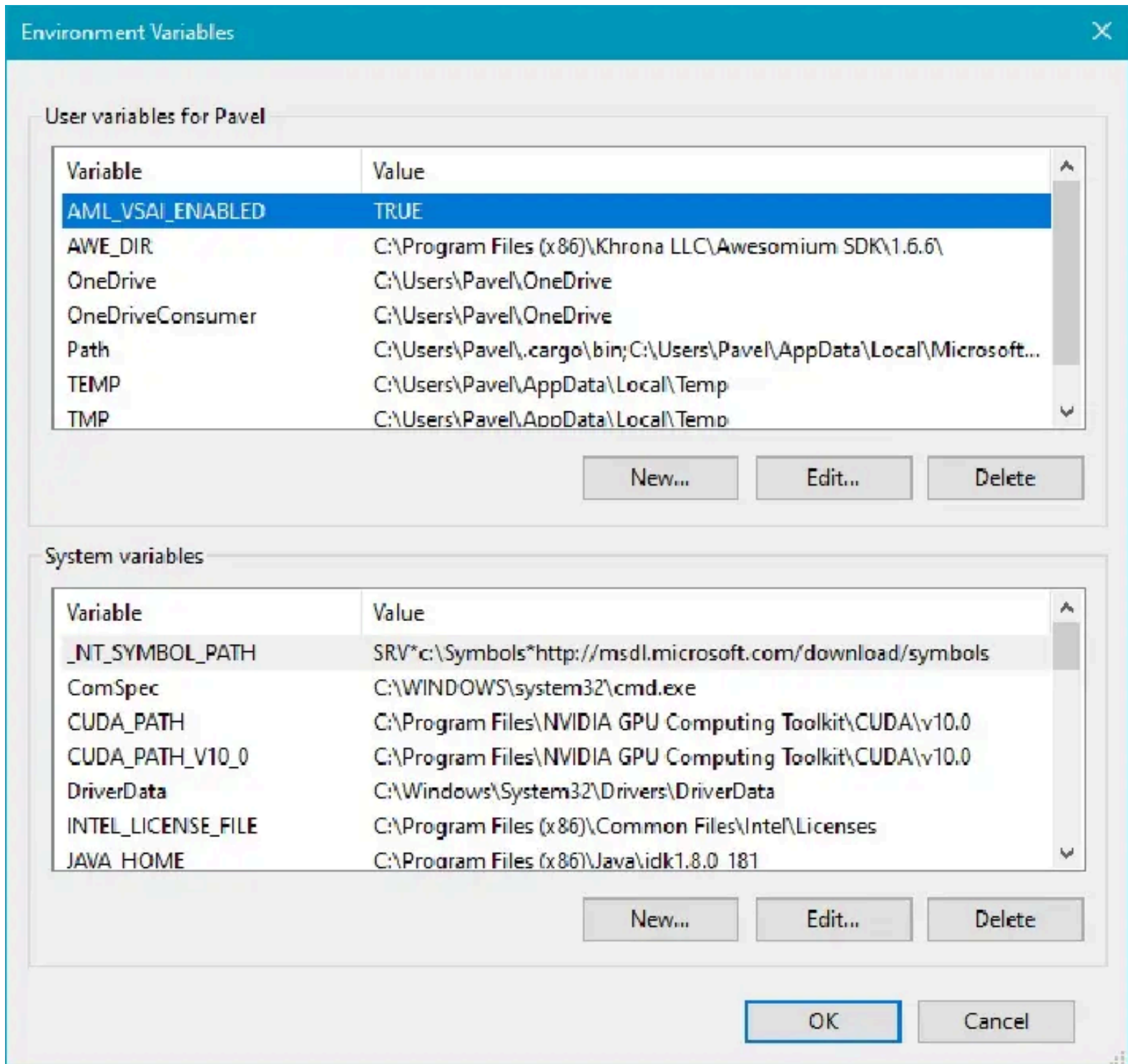


Figure 3-10: Environment variables editor



User environment variables are stored in *HKEY_CURRENT_USER\Environment*. System environment variables (applicable to all users) are stored at *HKEY_LOCAL_MACHINE\System\Current Control Set\Control\Session Manager\Environment*

A process receives environment variables from its parent process, which are a combination of system variables (applicable to all users) and user-specific variables. In most cases the environment variables a process receives are a copy of its parent's (see next section).

A console application can get the process environment variables with a third argument to `main` or `wmain`:

```
int main(int argc, char* argv[], const char* env[]);           // const is optional
int wmain(int argc, wchar_t* argv[], const wchar_t* env[]);    // const is optional
```

`env` is an array of string pointers, where the last pointer is `NULL`, signaling the end of the array. Each string is built in the following format:

name=value

The equals character separates the name from the value. The following example `main` function prints out the names and values of each environment variable:

```
int main(int argc, const char* argv[], char* env[]) {
    for (int i = 0; ; i++) {
        if (env[i] == nullptr)
            break;

        auto equals = strchr(env[i], '=');

        // change the equals to NULL
        *equals = '\\0';
        printf("%s: %s\\n", env[i], equals + 1);

        // for consistency, revert the equals sign
        *equals = '=';
    }
    return 0;
}
```

GUI applications can call `GetEnvironmentStrings` to get a pointer to an environment variables memory block, formatted like so:

```
name1=value1\0
name2=value2\0
...
\0
```

The following code snippet uses `GetEnvironmentStrings` to show all environment variables in one giant message box:

```
PWSTR env = ::GetEnvironmentStrings();
WCHAR text[8192] = { 0 };

auto p = env;
while (*p) {
    auto equals = wcschr(p, L'=');
    if (equals != p) { // eliminate empty names/values
        wcsncat_s(text, p, equals - p);
        wscat_s(text, L": ");
        wscat_s(text, equals + 1);
        wscat_s(text, L"\n");
    }
    p += wcslen(p) + 1;
}

::FreeEnvironmentStrings(env);
```

The environment block can be replaced in one swoop with `SetEnvironmentStrings` using the same format returned by `GetEnvironmentStrings`.

The environment block must be freed with `FreeEnvironmentStrings`. Normally, applications don't need to enumerate environment variables, but rather change or read a specific value. The following functions are used for this purpose:

```

BOOL SetEnvironmentVariable(
    _In_ LPCTSTR lpName,
    _In_opt_ LPCTSTR lpValue);

```

```

DWORD GetEnvironmentVariable(
    _In_opt_ LPCTSTR lpName,
    _Out_ LPTSTR lpBuffer,
    _In_ DWORD nSize);

```

GetEnvironmentVariable returns the number of characters copied to the buffer if it's large enough, or the length of the environment variable otherwise. It returns zero on failure (if the named variable does not exist). The usual practice is to call the function twice: first, with no buffer to get the length and then a second time after allocating a properly-sized buffer to receive the result. The following function can be used to get a variable's value by returning a C++ `std::wstring` as a result:

```

std::wstring ReadEnvironmentVariable(PCWSTR name) {
    DWORD count = ::GetEnvironmentVariable(name, nullptr, 0);
    if (count > 0) {
        std::wstring value;
        value.resize(count);
        ::GetEnvironmentVariable(name, const_cast<PWSTR>(value.data()), count);
        return value;
    }
    return L"";
}

```



The `const_cast` operator above removes the “constness” of `value.data()` as it returns `const wchar_t*`. A brutal C-style cast would work just as well: `(PWSTR)value.data()`.

Environment variables are used in many situations to specify information that is based on their current values. For example, a file path may be specified as “%windir%\explorer.exe”. The name between the percent characters is an environment variable that should be expended into its real value. Normal API functions don't have any special understanding of these intentions. Instead, the application must call `ExpandEnvironmentStrings` to convert any environment variable enclosed between percent signs to its value:

```
DWORD ExpandEnvironmentStrings(  
    _In_      LPCTSTR lpSrc,  
    _Out_opt_ LPTSTR  lpDst,  
    _In_      DWORD   nSize);
```

Just like `GetEnvironmentVariable`, `ExpandEnvironmentStrings` returns the number of characters copied to the target buffer or the number of characters that is needed if the buffer is too small (plus the NULL terminator). Here is an example usage:

```
WCHAR path[MAX_PATH];  
::ExpandEnvironmentStrings(L"%windir%\explorer.exe", path, MAX_PATH);  
printf("%ws\n", path); // c:\windows\explorer.exe
```

Creating Processes

Processes are created under the same user account with `CreateProcess`. Extended functions exist, such as `CreateProcessAsUser`, which will be discussed in chapter 16. `CreateProcess` requires an actual executable file. It cannot create a process based on a path to a document file. For example, passing something like `c:\MyData\data.txt`, assuming *data.txt* is some text file - will fail process creation. `CreateProcess` does not search for an associated executable to launch for *TXT* files. When a file is double-clicked in Explorer, for instance, a higher-level function from the Shell API is invoked - `ShellExecuteEx`. This function accepts any file, and if does not end with “EXE”, will search the registry based on the file extension to locate the associated program to execute. Then (if located), it will eventually call `CreateProcess`.

Where does Explorer look for these file associations? We'll look at that in chapter 17 (“Registry”)

`CreateProcess` accepts 9 parameters as follows:

```

BOOL CreateProcess(
    _In_opt_ PCTSTR pApplicationName,
    _Inout_opt_ PTSTR pCommandLine,
    _In_opt_ PSECURITY_ATTRIBUTES pProcessAttributes,
    _In_opt_ PSECURITY_ATTRIBUTES pThreadAttributes,
    _In_ BOOL bInheritHandles,
    _In_ DWORD dwCreationFlags,
    _In_opt_ PVOID pEnvironment,
    _In_opt_ PCTSTR pCurrentDirectory,
    _In_ PSTARTUPINFO pStartupInfo,
    _Out_ PPROCESS_INFORMATION lpProcessInformation);

```

The function returns `TRUE` on success, which means that from the kernel's perspective the process and the initial thread have been created successfully. It's still possible for the initialization done in the context of the new process (described in the previous section) to fail.

If successful, the real returned information is available via the last argument of type `PROCESS_INFORMATION`:

```

typedef struct _PROCESS_INFORMATION {
    HANDLE hProcess;
    HANDLE hThread;
    DWORD dwProcessId;
    DWORD dwThreadId;
} PROCESS_INFORMATION, *PPROCESS_INFORMATION;

```

There are four pieces of information provided: the unique process and thread IDs, and two open handles (with all possible permissions unless the new process is protected) to the newly created process and thread. Using the handles, the creating (parent) process can do anything it wants with the new process and thread (again, unless the process is protected, see later in this chapter). As usual, it's a good idea to close these handles once they are no longer needed.

Now let's turn our attention to the rest of the parameters that are inputs to the function.

pApplicationName and pCommandLine

These parameters should provide the executable path to run as a new process and any command-line arguments that are needed. These parameters are not interchangeable, however.

In most cases, you'll use the second argument for both the executable name and any command-line arguments needed to pass to the executable and set the first argument to `NULL`. Here are some of the benefits of the second argument compared to the first:

- If the file name has no extension, an `EXE` extension is added implicitly before searching for a match.
- If just a file name is supplied (not a full path) as the executable, the system searches in the directories listed in the previous section where the Loader looks for required DLLs, repeated here for convenience:
 1. The directory of the caller's executable
 2. The current directory of the process (discussed later in this section)
 3. The System directory returned by `GetSystemDirectory`
 4. The Windows directory returned by `GetWindowsDirectory`
 5. The directories listed in the `PATH` environment variable

If `pApplicationName` is not `NULL`, then it must be set to a full path to the executable. In that case, `pCommandLine` is still treated as command-line arguments.

One dent in the `pCommandLine` argument is that it's typed as `PTSTR`, meaning it's a non-const pointer to a string. This means `CreateProcess` actually writes (not just reads) to this buffer, which will cause an access violation if called with a constant string like so:

```
CreateProcess(nullptr, L"Notepad", ...);
```

Compile time static buffers are placed by default in a read-only section of the executable and mapped with the read only protection, causing any writes to raise an exception. The simplest solution is to place the string in read/write memory by building it dynamically or placing it on the stack (which is always read/write):

```
WCHAR name[] = L"Notepad";  
CreateProcess(nullptr, name, ...);
```

The final content of the buffer is the same as initially provided. You might be wondering why `CreateProcess` writes to the buffer. Unfortunately, there is no good reason that I know of and Microsoft should fix that. But they haven't for many years now, so I wouldn't

hold my breath.



This issue does not occur with `CreateProcessA` (the ASCII version of `CreateProcess`). The reason may be obvious: `CreateProcessA` must convert its arguments to Unicode, and for that it allocates a buffer dynamically (which is read/write), converts the string and then calls `CreateProcessW` with that allocated buffer. This does *not* mean you should use `CreateProcessA`!

pProcessAttributes and pThreadAttributes

These two parameters are `SECURITY_ATTRIBUTES` pointers (for the newly created process and thread), discussed in chapter 2. In most cases, `NULL` should be passed in, unless the returned handles should be inheritable and in that case an instance with `bInheritHandle = TRUE` may be passed in.

bInheritHandles

This parameter is a global switch that allows or disallows handle inheritance (described in the next sub-section). If `FALSE`, no handles from the parent process are inherited by the (newly created) child process. If `TRUE`, all handles that are marked inheritable will be inherited by the child process.

dwCreationFlags

This parameter can be a combination of various flags, the more useful ones described in table 3-5. Zero is a reasonable default in many cases.

Table 3-5: Some flags for `CreateProcess`

Flag	Description
<code>CREATE_BREAKAWAY_FROM_JOB</code>	If the parent process is part of a job, the child process is not, unless the job does not allow breaking out of it in which case the child process is still created under the same job (see next chapter for more on jobs)
<code>CREATE_SUSPENDED</code>	The process and thread are created, but the thread is suspended. The parent process eventually should call <code>ResumeThread</code> on the returned thread handle to start execution

Table 3-5: Some flags for `CreateProcess`

Flag	Description
<code>DEBUG_PROCESS</code>	The parent process becomes a debugger, and the created process is the debuggee. The debugger will start getting debugging events related to the child process. Any processes created from the child also become debuggees under the control of the parent process
<code>DEBUG_ONLY_THIS_PROCESS</code>	Similar to <code>DEBUG_PROCESS</code> , but only the child process becomes a debuggee, rather than all child processes created by the child process
<code>CREATE_NEW_CONSOLE</code>	The new process gets its own console (if it's a CUI application) rather than inheriting its parent console
<code>CREATE_NO_WINDOW</code>	If the child is a CUI application, it's created without a console
<code>DETACHED_PROCESS</code>	Sort of the opposite of <code>CREATE_NEW_CONSOLE</code> . The child process does not get any console. If it needs one, it can call <code>AllocConsole</code> to create one
<code>CREATE_PROTECTED_PROCESS</code>	The new process must be run protected (see later in this chapter)
<code>CREATE_PROTECTED_PROCESS</code>	Create the new process as protected. This only works for executables that are signed by Microsoft specifically for this
<code>CREATE_UNICODE_ENVIRONMENT</code>	Creates the environment block for the new process as Unicode rather than the default (which is ironically ASCII)
<code>INHERIT_PARENT_AFFINITY</code>	(Windows 7+) The child process inherits its parent group affinity (see chapter 6 for more on affinity)
<code>EXTENDED_STARTUPINFO_PRESENT</code>	The process is created with an extended <code>STARTUPINFOEX</code> structure that contains process attributes (see the section "Process (and Thread) Attributes" later in this chapter)
<code>CREATE_DEFAULT_ERROR_MODE</code>	Creates the process with the system default error mode rather than inheriting it from the parent. See the section on Error Mode in chapter 20

In addition to the flags in table 3-5, the creator can set the process priority class, based on table 3-6.

Table 3-6: Priority class flags in `CreateProcess`

Priority class flag	Base priority value
<code>IDLE_PRIORITY_CLASS</code>	4
<code>BELOW_NORMAL_PRIORITY_CLASS</code>	6
<code>NORMAL_PRIORITY_CLASS</code>	8
<code>ABOVE_NORMAL_PRIORITY_CLASS</code>	10
<code>HIGH_PRIORITY_CLASS</code>	13
<code>REALTIME_PRIORITY_CLASS</code>	24

If no priority class flag is specified, the default is Normal unless the creator's priority class is Below Normal or Idle, in which case the new process inherits its parent's priority class. If the Real-time priority class is specified, the child process must execute with admin privileges; otherwise, it gets a High priority class instead.

The priority class has little meaning for the process itself. Rather, it sets the default priority for threads in the new process. We'll look at the effects of priorities in chapter 6.

pEnvironment

This is an optional pointer to an environment variables block to be used by the child process. Its format is the same as returned by `GetEnvironmentStrings` discussed earlier in this chapter. In most cases, `NULL` is passed in, which causes the parent's environment block to be copied to the new process' environment block.

pCurrentDirectory

This sets the current directory for the new process. The current directory is used as part of the search for files in case a file name only is used rather than a full path. For example, a call to the `CreateFile` function with a file named "mydata.txt", will search for the file in the process' current directory. The `pCurrentDirectory` parameter allows the parent process to set the current directory for the created process which can affect the locations in which DLL search is performed for required DLLs. In most cases, `NULL` is passed in, which sets the current directory of the new process to the current directory of the parent.

Normally, a process can change its current directory with `SetCurrentDirectory`. Note this is a process-wide rather than a thread setting:

```
BOOL SetCurrentDirectory(
    _In_ PCTSTR pPathName);
```

The current directory consists of a drive letter and path or a share name in the *Universal Naming Convention* (UNC), such as `\\MyServer\MyShare`.

Naturally, the current directory can be queried back with `GetCurrentDirectory`:

```
DWORD GetCurrentDirectory(
    _In_  DWORD  nBufferLength,
    _Out_ LPTSTR lpBuffer);
```

The return value is zero on failure, or the number of characters copied to the buffer (including the NULL terminator). If the buffer is too small, the returned value is the required character length (including the NULL terminator).

pStartupInfo

This parameter points to one of two structures, `STARTUPINFO` or `STARTUPINFOEX`, defined like so:

```
typedef struct _STARTUPINFO {
    DWORD cb;
    PTSTR lpReserved;
    PTSTR lpDesktop;
    PTSTR lpTitle;
    DWORD dwX;
    DWORD dwY;
    DWORD dwXSize;
    DWORD dwYSize;
    DWORD dwXCountChars;
    DWORD dwYCountChars;
    DWORD dwFillAttribute;
    DWORD dwFlags;
    WORD  wShowWindow;
    WORD  cbReserved2;
    PBYTE lpReserved2;
    HANDLE hStdInput;
    HANDLE hStdOutput;
    HANDLE hStdError;
} STARTUPINFO, *PSTARTUPINFO;

typedef struct _STARTUPINFOEX {
    STARTUPINFO StartupInfo;
    PPROC_THREAD_ATTRIBUTE_LIST pAttributeList;
} STARTUPINFOEX, *LPSTARTUPINFOEX;
```

The minimum usage for this parameter is to create a `STARTUPINFO` structure, initialize its size (cb member) and zero out the rest. Zeroing the structure is important, otherwise it holds junk values which will likely cause `CreateProcess` to fail. Here is the minimal code:

```
STARTUPINFO si = { sizeof(si) };
CreateProcess(..., &si, ...);
```



`STARTUPINFOEX` is discussed in the section “Process (and Thread) Attributes” later in this chapter.

The `STARTUPINFO` and `STARTUPINFOEX` structures provide more customization options for process creation. Some of their members are only used if certain values are set in the `dwFlags` member (in addition to other flags). Table 3-7 details `dwFlags` possible values and their meaning.

Table 3-7: The `dwFlags` member of `STARTUPINFO` flags

Flag	Meaning
<code>STARTF_USESHOWWINDOW</code>	The <code>wShowWindow</code> member is valid
<code>STARTF_USESIZE</code>	The <code>dwXSize</code> and <code>dwYSize</code> members are valid
<code>STARTF_USEPOSITION</code>	The <code>dwX</code> and <code>dwY</code> members are valid
<code>STARTF_USECOUNTCHARS</code>	The <code>dwXCountChars</code> and <code>dwYCountChars</code> members are valid
<code>STARTF_USEFILLATTRIBUTE</code>	The <code>dwFillAttribute</code> member is valid
<code>STARTF_RUNFULLSCREEN</code>	For console apps, run full screen (x86 only)
<code>STARTF_FORCEONFEEDBACK</code>	Instructs Windows to show the “Working in Background” cursor, whose shape can be found in the Mouse Properties dialog shown in figure 3-11. If during the next 2 seconds, the process makes GUI calls, it gives the process an additional 5 seconds with this cursor showing. If at any point, the process calls <code>GetMessage</code> , indicating it’s ready to process messages, the cursor is immediately reverted to normal.
<code>STARTF_FORCEOFFFEEDBACK</code>	Does not show the “Working in Background” cursor
<code>STARTF_USESTDHANDLES</code>	The <code>hStdInput</code> , <code>hStdOutput</code> and <code>hStdError</code> members are valid
<code>STARTF_USEHOTKEY</code>	The <code>hStdInput</code> member is valid and is a value sent as <code>wParam</code> for a <code>WM_HOTKEY</code> message. Refer to the documentation for more information
<code>STARTF_TITLEISLINKNAME</code>	The <code>lpTitle</code> member is a path to a shortcut file (<code>.lnk</code>) used to start the process. The Shell (Explorer) sets this appropriately
<code>STARTF_TITLEISAPPID</code>	The <code>lpTitle</code> member is an <i>AppUserModelId</i> . See discussion after this table

Table 3-7: The `dwFlags` member of `STARTUPINFO` flags

Flag	Meaning
<code>STARTF_PREVENTPINNING</code>	Prevents any windows created by the process from being pinned to the task bar. Only works if <code>STARTF_TITLEISAPPID</code> is also specified
<code>STARTF_UNTRUSTEDSOURCE</code>	Indicates the command line passed to the process is from an untrusted source. This is a hint to the process to check its command line carefully

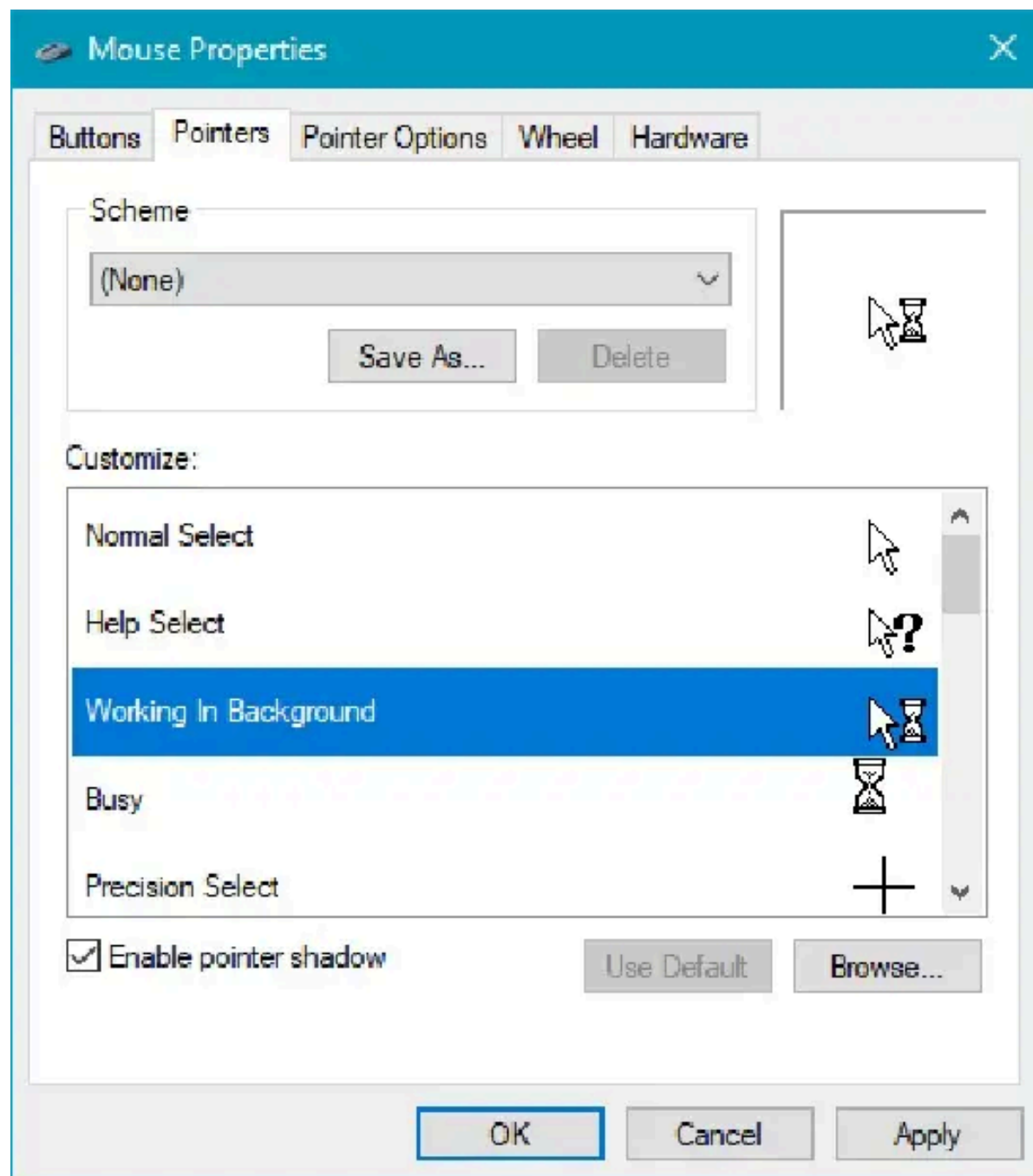


Figure 3-11: “Working in Background” mouse cursor

Let’s now examine the other members of `STARTUPINFO`.

There are three reserved members, `lpRerved`, `lpReserved2` and `cbReserved2` - these should be set to `NULL`, `NULL` and zero, respectively.

The `lpDesktop` member specifies an alternate Window Station for the new process and an alternate Desktop for the new thread. If this member is `NULL` (or an empty string), the parent process Window Station and Desktop are used. Alternatively, a full desktop name can be specified in the format *windowstation\desktop*. For example “winsta0\mydesktop” could be used. See the sidebar *Window Stations and Desktops* for more information.

Window Stations and Desktops

A Window Station is a kernel object which is part of a session. It contains user related objects: A clipboard, an atom table and desktops. A desktop contains windows, menus and hooks. A process is associated with a single Window Station. The interactive Window Station is always named “WinSta0” and is the only one within a session that can be “interactive”, meaning used with input devices.

By default, a interactive logon session has a Window Station named “winsta0”, in which two desktops exist: the “default” desktop, where the user normally works - where you see Explorer, the task bar and whatever else you normally run. Another desktop (created by the *Winlogon.exe* process is called “Winlogon” and is the one used when pressing the famous Ctrl+Alt+Del key combination. Windows calls the `SwitchDesktop` function to switch the input desktop to the “Winlogon” desktop. Desktops can be created or opened with `CreateDesktop` and `OpenDesktop`, respectively.

You can find more details in my blog post at <https://scorpiosoftware.net/2019/02/17/windows-10-desktops-vs-sysinternals-desktops/>.

The `lpTitle` member can hold the title for console applications. If `NULL`, the executable name is used as the title. If `dwFlags` has the flag `STARTF_TITLEISAPPID` (Windows 7 and later), then `lpTitle` is an `AppUserModelId`, which is a string identifier that the shell uses for task bar item grouping and jump lists. Processes can set their `AppUserModelId` explicitly by calling `SetCurrentProcessExplicitAppUserModelID` rather than letting their parent dictate it. Working with jump lists and other task bar features are beyond the scope of this book.

`dwX` and `dwY` can be set as default location for windows created by the process. They are used only if `dwFlags` includes `STARTF_USEPOSITION`. The new process can use these values if its calls to `CreateWindow` or `CreateWindowEx` use `CW_USEDEFAULT` for the window’s position. (See the `CreateWindow` function documentation for more details.) `dwXSize` and `dwYSize` are

similar, specifying default width and height of new windows created by the child process if it uses `CW_USEDEFAULT` as width and height in calls to `CreateWindow/CreateWindowEx`. (Of course `STARTF_USESIZE` must be set in `dwFlags` for these values to propagate).

`dwXCountChars` and `dwYCountChars` set the initial width and height (in characters) of a console window created by the child process (if any). As with the previous members, `dwFlags` must have `STARTF_USECOUNTCHARS` for these values to have any effect.

`dwFillAttribute` specifies the initial text and background colors if a new console window is created with the process. As usual, this member has effect if `dwFlags` includes `STARTF_USEFILLATTRIBUTE`. The possible color combinations have 4 bits each, leading to 16 combinations for text and background. The possible values are shown in table 3-8.

Table 3-8: Color values for consoles

Color constant	Value	Text/background
<code>FOREGROUND_BLUE</code>	0x01	Text
<code>FOREGROUND_GREEN</code>	0x02	Text
<code>FOREGROUND_RED</code>	0x04	Text
<code>FOREGROUND_INTENSITY</code>	0x08	Text
<code>BACKGROUND_BLUE</code>	0x10	Background
<code>BACKGROUND_GREEN</code>	0x20	Background
<code>BACKGROUND_RED</code>	0x40	Background
<code>BACKGROUND_INTENSITY</code>	0x80	Background

`wShowWindow` (valid if `dwFlags` includes `STARTF_USESHOWWINDOW`) indicates the way the main window should be shown by the process (assuming it has a GUI). These are values normally passed to the `ShowWindow` function, with the `SW_` prefix. `wShowWindow` is unique because it's provided directly in the `WinMain` function as the last parameter. Of course the created process can just disregard the provided value and show its windows in any way it sees fit. But it's a good practice to honor this value. If the creator does not provide this member, `SW_SHOWDEFAULT` is used, indicating the application can use any logic in displaying its main window. For example, it may have saved the last window position and state (maximized, minimized, etc.) and so it will restore the window to the saved position/state.

One scenario where this value is controllable is with shortcuts created using the shell. Figure 3-12 shows a shortcut created for running *Notepad*. In the shortcut properties, the way the initial window is shown can be selected: normal, minimized or maximized. Explorer propagates this value in the `wShowWindow` member when creating the process (`SW_SHOWNORMAL`, `SW_SHOWMINNOACTIVE`, `SW_SHOWMAXIMIZED`).

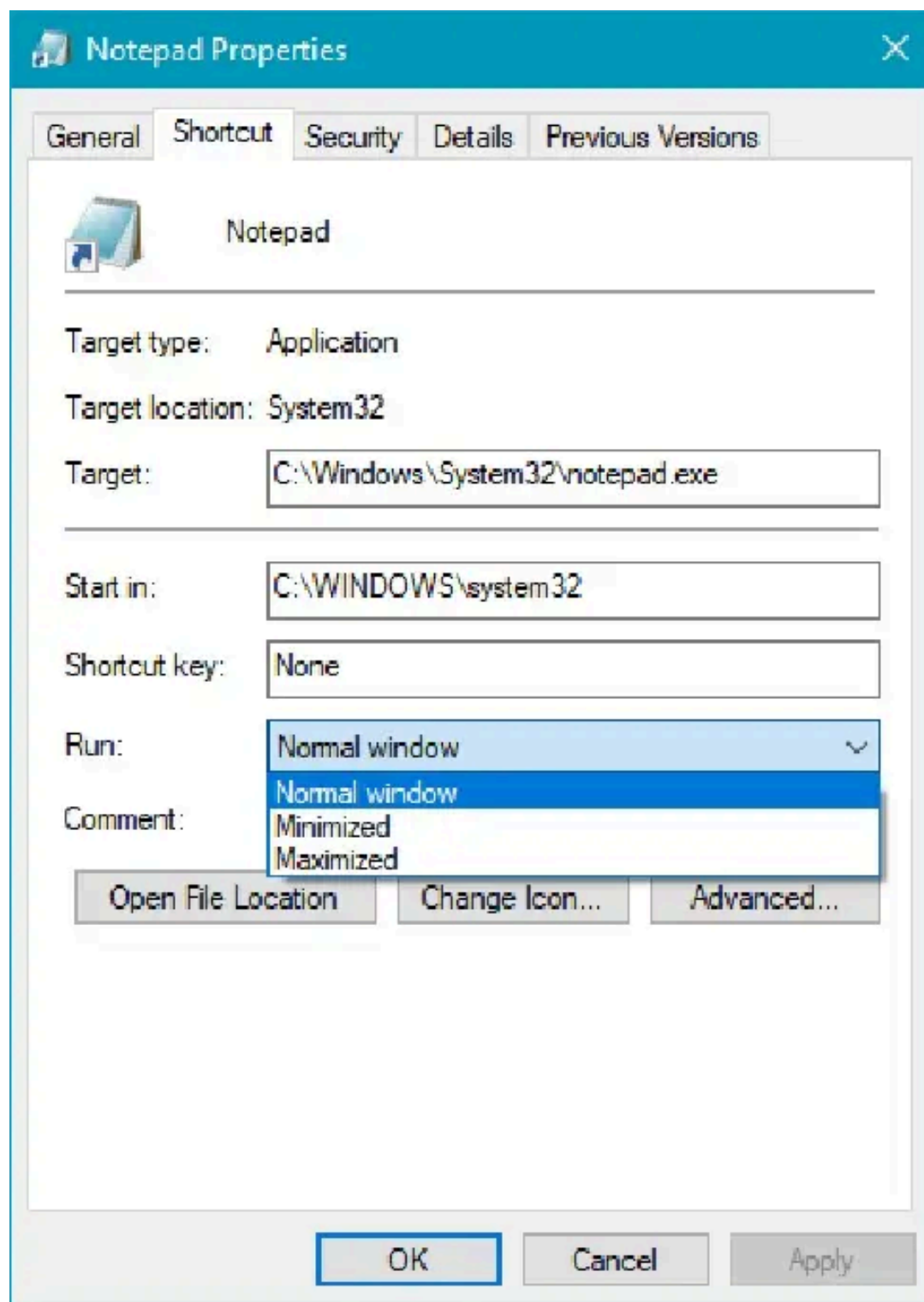


Figure 3-12: Show window in shortcuts

The last three members in `STARTUPINFO` are handles to standard input (`hStdInput`), output (`hStdOutput`) and error (`hStdError`). If `dwFlags` contains `STARTF_USEHANDLES`, these handles will be used in the new process as such. Otherwise, the new process will have the defaults: input from keyboard, output and error to the console buffer.

If the process is launched from the taskbar or a jump list (Windows 7+), the `hStdOutput` handle is actually a handle to a monitor (HMONITOR) from which the process was launched.

Given all these various options for process creation (and there are more discussed in the section “Process (and Thread) Attributes” later in this chapter), it may seem daunting to create a process, but in most cases it’s fairly straightforward if the defaults are acceptable. The following code snippet creates an instance of *Notepad*:

```
WCHAR name[] = L"notepad";
STARTUPINFO si = { sizeof(si) };
PROCESS_INFORMATION pi;

BOOL success = ::CreateProcess(nullptr, name, nullptr, nullptr, FALSE,
    0, nullptr, nullptr, &si, &pi);
if (!success) {
    printf("Error creating process: %d\n", ::GetLastError());
}
else {
    printf("Process created. PID=%d\n", pi.dwProcessId);
    ::CloseHandle(pi.hProcess);
    ::CloseHandle(pi.hThread);
}
```

What can be done with the returned handles from `CreateProcess`? One thing is being notified when the process terminated (for whatever reason). This is done with the `WaitForSingleObject` function. This function is not specific to a process, but can wait for various kernel objects until they become **signaled**. The meaning of *signaled* depends on the object type; for a process, it means terminated. A detailed discussion of the “wait” functions is saved for chapter 8. Here, we’ll look at a couple of examples. First we can wait indefinitely until the process exits:

```
// process creation succeeded
printf("Process created. PID=%d\n", pi.dwProcessId);
::WaitForSingleObject(pi.hProcess, INFINITE);
printf("Notepad terminated.\n");

::CloseHandle(pi.hProcess);
::CloseHandle(pi.hThread);
}
```

`WaitForSingleObject` puts the calling thread into a wait state until the object in question changes to the *signaled* state or the timeout expires. In case of `INFINITE` (-1), it never expires. Here is an example for a non-`INFINITE` timeout:

```
DWORD rv = ::WaitForSingleObject(pi.hProcess, 10000);    10 seconds
if (rv == WAIT_TIMEOUT)
    printf("Notepad still running...\n");
else if (rv == WAIT_OBJECT_0)
    printf("Notepad terminated.\n");
else // WAIT_ERROR (unlikely in this case)
    printf("Error! %d\n", ::GetLastError());
```

The calling thread blocks for no more than 10000 milliseconds, after which the returned value indicates the state of the process.



A process can always get the `STARTUPINFO` it was created with by calling `GetStartupInfo`.

Handle Inheritance

In chapter 2 we looked at ways to share kernel objects between processes. One is sharing by name the other by duplicating handles. The third option is to use handle inheritance. This option is only available if a process creates a child process. At the point of creation, the parent process can duplicate a selected set of handles to the target process. Once `CreateProcess` is called with the fifth argument set to `TRUE`, all handles in the parent process that have their inheritance bit set will be duplicated into the child process, where the handle values are the same as in the parent process.

The last sentence is important. The child process cooperates with the parent process (presumably they are part of the same software system), and it knows it's going to get some handles from its parent. What it does *not* know is what the values of these handles are. One simple way of providing these values is using the command line arguments sent to the process being created.

Setting a handle to be inheritable can be done in several ways:

- If the object in question is created by the parent process, then its `SECURITY_ATTRIBUTES` can be initialized with a handle inheritance flag and passed to the *Create* function like so:

```
SECURITY_ATTRIBUTES sa = { sizeof(sa) };
sa.bInheritHandles = TRUE;

HANDLE h = ::CreateEvent(&sa, FALSE, FALSE, nullptr);

// handle h will be inherited by child processes
```

- For an existing handle, call `SetHandleInformation`:

```
::SetHandleInformation(h, HANDLE_FLAG_INHERIT, HANDLE_FLAG_INHERIT);
```

- Lastly, most *Open* functions allow setting the inheritance flag on a successfully returned handle. Here is an example for a named event object:

```
HANDLE h = ::OpenEvent(EVENT_ALL_ACCESS,
    TRUE,    // inheritable
    L"MyEvent");
```

The *InheritSharing* application is yet another variation on the memory sharing applications from chapter 2. This time, the sharing is achieved by inheriting the memory mapping handle to child processes created from a first process. The dialog now has an extra *Create* button to spawn new processes with an inherited shared memory handle (figure 3-13).

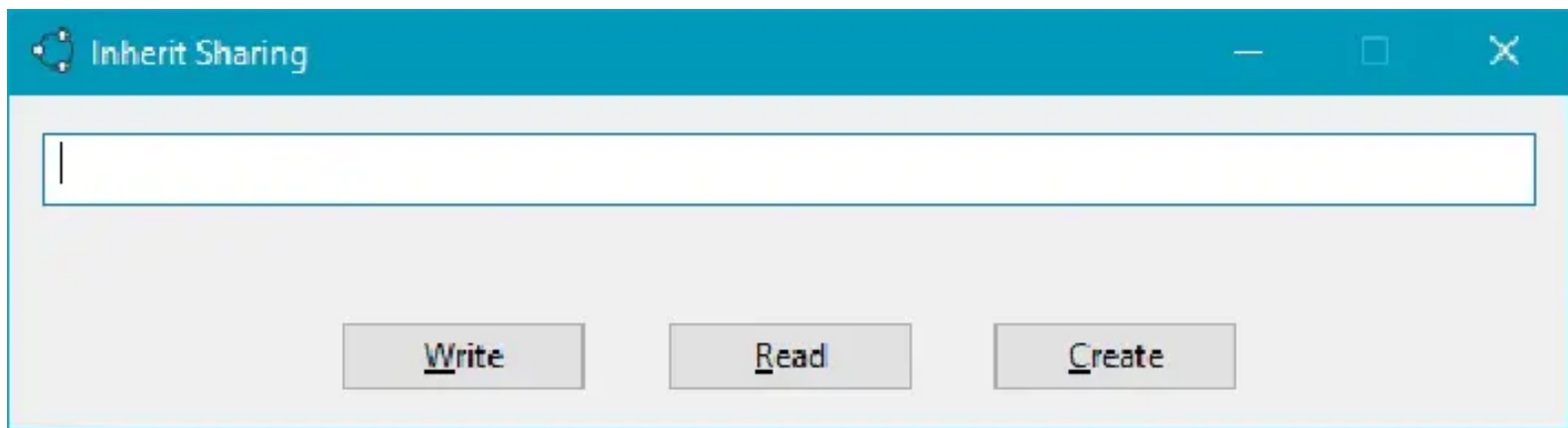


Figure 3-13: Sharing by Inheritance application

A *InheirtSharing* process creates another instance of itself when the *Create* button is clicked. The new instance must get a handle to the shared memory object, and this is done by inheritance: the existing shared memory handle (held in a `wil::unique_handle` object) needs to be made inheritable so that it can be duplicated to the new process. The *Create* button click handler starts with setting the inheritance bit:

```
::SetHandleInformation(m_hSharedMem.get(), HANDLE_FLAG_INHERIT, HANDLE_FLAG_INH\
ERIT);
```

Now the new process can be created with the fifth argument set to `TRUE`, indicating all inheritable handles are to be duplicated for the new process. In addition, the new process needs to know the value of its duplicated handle, and this is passed in the command line:

```
STARTUPINFO si = { sizeof(si) };
PROCESS_INFORMATION pi;

// build command line
WCHAR path[MAX_PATH];
::GetModuleFileName(nullptr, path, MAX_PATH);
WCHAR handle[16];
::_itow_s((int)(ULONG_PTR)m_hSharedMem.get(), handle, 10);
::wcscat_s(path, L" ");
::wcscat_s(path, handle);

// now create the process

if (::CreateProcess(nullptr, path, nullptr, nullptr, TRUE,
    0, nullptr, nullptr, &si, &pi)) {
```

```

    // close unneeded handles
    ::CloseHandle(pi.hProcess);
    ::CloseHandle(pi.hThread);
}
else {
    MessageBox(L"Failed to create new process", L"Inherit Sharing");
}

```

The command line is built by first calling `GetModuleFileName`, which generally allows getting a full path for any DLL loaded in the process. With the first argument set to `NULL`, the executable full path is returned. This approach is robust, such that there is no dependency on the actual location of the executable in the file system.

Once this path is returned, the handle value is appended as a command-line argument. Remember that an inherited handle always has the same value as in the original process. This is possible because the new process handle table is initially empty, so the entry is definitely unused.

The last piece of the puzzle is when the process starts up. It needs to know whether it's the first instance, or an instance that gets an existing inherited handle. In the `WM_INITDIALOG` message handler, the command line needs to be examined. If there is no handle value in the command line then the process needs to create the shared memory object. Otherwise, it needs to grab the handle and just use it.

```

int count;
PWSTR* args = ::CommandLineToArgvW(::GetCommandLine(), &count);
if (count == 1) {
    // "master" instance
    m_hSharedMem.reset(::CreateFileMapping(INVALID_HANDLE_VALUE,
        nullptr, PAGE_READWRITE, 0, 1 << 16, nullptr));
}
else {
    // first "real" argument is inherited handle value
    m_hSharedMem.reset((HANDLE)(ULONG_PTR)::_wtoi(args[1]));
}
::LocalFree(args);

```

Since this is not `WinMain`, the command line arguments are not readily available. `GetCommandLine` can always be used to get the command line at any time. Then `CommandLineToArgvW` is used to parse the arguments (discussed earlier in this chapter). If no handle value

is passed in, `CreateFileMapping` is used to create the shared memory. Otherwise, the value is interpreted as a handle and attached to the `wil::unique_handle` object for safe keeping.

You can try creating a new instance from a child process - it works in exactly the same way as using the “original” handle to propagate to the child process.

Debugging Child Processes with Visual Studio

In the *InheritSharing* application, it’s desirable to debug not just the main instance, but also a child process, because it’s initiated with a different command line. Visual Studio by default does not debug child processes (processes created by the debugged process).

There is, however, an extension to Visual Studio that allows this. Open the extensions dialog (*Tools/Extensions and Updates* in VS 2017, *Extensions/Manage Extensions* in VS 2019), go to the *Online* node and search for *Microsoft Child Process Debugging Power Tool* and install it (figure 3-14).



Figure 3-14: Child Process Debugging Power Tool in extensions

Once installed, go to *Debug/Other Debug Targets/ChildProcess Debugger Settings...*, check *Enable ChildProcess Debugging* and click *Save*. Now set a breakpoint at `CMainDlg::OnInitDialog` and start debugging normally (F5).

The first time you hit the breakpoint is when the dialog comes up for a fresh process, creating its own shared memory object. The count variable should be 1 (figure 3-15).

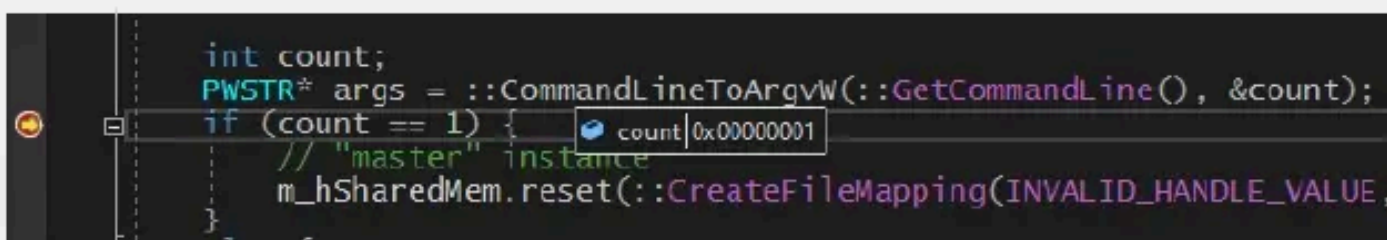


Figure 3-15: Breakpoint in the first process

Continue debugging and click *Create*. A new process should come up under the control of the debugger and the same breakpoint should hit again (figure 3-16). Notice count is now 2. Also notice it’s a different process - the Processes toolbar combobox should show two

processes (figure 3-17).

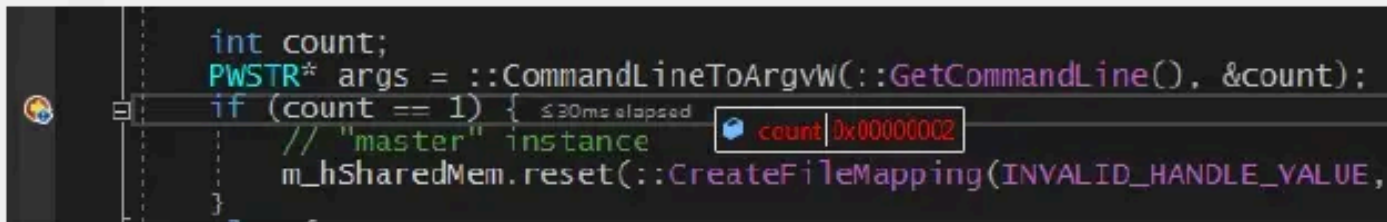


Figure 3-16: Breakpoint in the second process

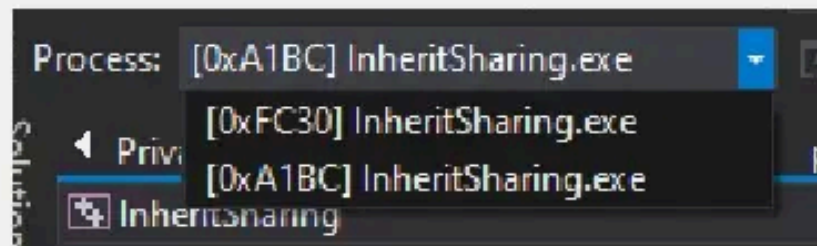


Figure 3-17: Debugging multiple processes

Process Drive Directories

Every process has its current directory, set with `SetCurrentDirectory` and retrieved with `GetCurrentDirectory`. This directory is used when accessing a file without any path prefix such as “mydata.txt”. What about default directories when accessing a file with a drive prefix like “c:mydata.txt” (notice the lack of a backslash).

As it turns out, the system keeps track of the current directory for each drive using process environment variables. If you call `GetEnvironmentStrings`, you’ll discover something like the following at the beginning of the block:

```
=C:=C:\Dev\Win10SysProg
=D:=D:\Temp
```

To get the current directory for a drive call `GetFullPathName`:

```
DWORD GetFullPathNameW(
    _In_ LPCWSTR lpFileName,
    _In_ DWORD nBufferLength,
    _Out_ LPWSTR lpBuffer,
    _Outptr_opt_ LPWSTR* lpFilePart);
```

Generally speaking, this function returns the full path of a given file name. Specifically, with a drive letter it returns its current directory. Here is an example:

```
WCHAR path[MAX_PATH];
::GetFullPathName(L"c:", MAX_PATH, path, nullptr);
```



Do not append a backslash to the drive letter after the colon! If you do, you'll just get the same string back.

Calling the function with a drive letter and a file name returns the resulting full path of the drive's current directory and the file name:

```
WCHAR path[MAX_PATH];
::GetFullPathName(L"c:mydata.txt", MAX_PATH, path, nullptr);    // no backslash
```

The above code might return something like “c:Win10SysProg\mydata.txt”.



GetFullPathName does not check for the existence of the file provided.

Process (and Thread) Attributes

The STARTUPINFO structure we met in CreateProcess has quite a few fields in it. It stands to reason that future versions of Windows may require more ways to customize process creation. One possible way would be to extend the STARTUPINFO structure and add more flags to make certain members valid. Microsoft decided to extend STARTUPINFO in a different way starting with Windows Vista.

An extended structure, STARTUPINFOEX is defined, that extends STARTUPINFO, shown here again for convenience:

```
typedef struct _STARTUPINFOEX {
    STARTUPINFO StartupInfo;
    PPROC_THREAD_ATTRIBUTE_LIST pAttributeList;
} STARTUPINFOEXW, *LPSTARTUPINFOEXW;
```

The memory layout of `STARTUPINFOEX` starts with a `STARTUPINFO` with just one added member: an opaque attribute list. This attribute list is the main extension mechanism for `CreateProcess` (and `CreateRemoteThreadEx` discussed in chapter 5). Since this attribute list can point to any number of attributes, there is no need to extend `STARTUPINFOEX` further.

Creating and filling an attribute list requires the following steps:

1. Allocate and initialize an attribute list with `InitializeProcThreadAttributeList`.
2. Add attributes as required by calling `UpdateProcThreadAttribute` once for each attribute.
3. Set the `pAttribute` member of `STARTUPINFOEX` to point to the attribute list.
4. Call `CreateProcess` with the extended structure, not forgetting to add the flag `EXTENDED_STARTUPINFO_PRESENT` to the creation flags (sixth parameter to `CreateProcess`)
5. Delete the attribute list with `DeleteProcThreadAttributeList`.

Let's take these steps in turn. Here is the simplified declaration of `InitializeProcThreadAttributeList`:

```
BOOL InitializeProcThreadAttributeList(
    _Out_ PPROC_THREAD_ATTRIBUTE_LIST pAttributeList,
    _In_ DWORD dwAttributeCount,
    _Reserved_ DWORD dwFlags,    // must be zero
    PSIZE_T pSize);
```

The first step is to allocate a buffer large enough to hold the required number of attributes. This is done by calling `InitializeProcThreadAttributeList` twice: first to get the required size, allocate a buffer, and then make a second call to initialize the buffer to hold an attribute list.

The following example performs these steps (error handling omitted):

```

SIZE_T size;
// get required size
::InitializeProcThreadAttributeList(nullptr, 1, 0, &size);

// allocate the required size
auto attlist = (PPROC_THREAD_ATTRIBUTE_LIST)malloc(size);

// initialize
::InitializeProcThreadAttributeList(attlist, 1, 0, &size); // just one attribute

```

The first call to `InitializeProcThreadAttributeList` returns `FALSE`, with `GetLastError` returning 122 (“The data area passed to a system call is too small.”). This is expected, since the real return value is the required size. The attribute list itself must be allocated by the caller (`malloc` is used in the above snippet), which also means it must be freed after the call to `CreateProcess`.

Next, `UpdateProcThreadAttribute` is called a number of times according to the count of attributes. The list of possible attributes has grown with almost every Windows release, and is likely to continue growing. Table 3-9 shows the *documented* attributes for processes and threads (at the time of writing), with a brief description.

Table 3-9: Documented process and thread attributes

Attribute constant	Applies to	Minimum version	Description
PROC_THREAD_ATTRIBUTE			
PARENT_PROCESS	Process	Windows Vista	Sets a different parent process from which to inherit various properties
HANDLE_LIST	Process	Windows Vista	Specifies a list of handles to be inherited by the child process
GROUP_AFFINITY	Thread	Windows 7	Sets the default CPU affinity group for the new thread (see chapter 6)
PREFERRED_NODE	Process	Windows 7	Sets the preferred NUMA node for the new process
IDEAL_PROCESSOR	Thread	Windows 7	Sets the ideal CPU for the new thread (see chapter 6)

Table 3-9: Documented process and thread attributes

Attribute constant	Applies to	Minimum version	Description
PROC_THREAD_ATTRIBUTE			
UMS_THREAD	Thread	Windows 7	Sets the User Mode Scheduling (UMS) context for the new thread (see chapter 10)
MITIGATION_POLICY	Process	Windows 7	Sets security mitigation policies for the new process (see chapter 16)
SECURITY_CAPABILITIES	Process	Windows 8	Sets the security capabilities of an AppContainer (see chapter 16)
PROTECTION_LEVEL	Process	Windows 8	Launches the new process with the same protection level as the creator
CHILD_PROCESS_POLICY	Process	Windows 10	Specifies whether the new process can create child processes
DESKTOP_APP_POLICY	Process	Windows 10 (1703)	Applies to applications converted to UWP using <i>Desktop Bridge</i> . Specifies whether the new process' child processes will be created outside the desktop app environment

Desktop Bridge is discussed in chapter 18.

UpdateProcThreadAttribute is defined like so:

```

BOOL UpdateProcThreadAttribute(
    _Inout_ PPROC_THREAD_ATTRIBUTE_LIST pAttributeList,
    _In_ DWORD dwFlags,           // must be zero
    _In_ DWORD_PTR Attribute,
    _In_ PVOID pValue,
    _In_ SIZE_T cbSize,
    _Out_ PVOID pPreviousValue,   // must be NULL
    _In_opt_ PSIZE_T pReturnSize); // must be NULL

```

The following example uses `PROC_THREAD_ATTRIBUTE_PARENT_PROCESS` attribute to set a different parent by specifying a handle to another process:

```

HANDLE hParent = ...;
::UpdateProcThreadAttribute(attlist, 0, PROC_THREAD_ATTRIBUTE_PARENT_PROCES\
S,
    &hParent, sizeof(hParent), nullptr, nullptr);

```

With `PROC_THREAD_ATTRIBUTE_PARENT_PROCESS`, the attribute value is an open handle to the relevant process. See the documentation for details on the other attributes values.

Once the attribute list is fully updated, the call to `CreateProcess` can commence, being careful to use the correct structure and flags for the attributes to have any effect:

```

STARTUPINFOEX si = { sizeof(si) };
si.lpAttributeList = attlist;

PROCESS_INFORMATION pi;
WCHAR name[] = L"Notepad";

::CreateProcess(nullptr, name, nullptr, nullptr, FALSE,
    EXTENDED_STARTUPINFO_PRESENT, nullptr, nullptr, (STARTUPINFO*)&si, &pi);

```

There are two things that must be set to indicate an attribute list: the `STARTUPINFOEX` structure and the flag `EXTENDED_STARTUPINFO_PRESENT`. Without the latter, the attributes won't be applied.

The last step is cleaning up the attribute list and the allocated memory for it:

```

::DeleteProcThreadAttributeList(attList);
::free(attList);

```

Given the above steps, the following function creates a given process parented with another process based on its ID:

```

DWORD CreateProcessWithParent(PWSTR name, DWORD parentPid) {
    HANDLE hParent = ::OpenProcess(PROCESS_CREATE_PROCESS, FALSE, parentPid);
    if (!hParent)
        return 0;

    PROCESS_INFORMATION pi = { 0 };
    PPROC_THREAD_ATTRIBUTE_LIST attList = nullptr;

    do {
        SIZE_T size = 0;
        ::InitializeProcThreadAttributeList(nullptr, 1, 0, &size);
        if (size == 0)
            break;

        attList = (PPROC_THREAD_ATTRIBUTE_LIST)malloc(size);
        if (!attList)
            break;

        if (!::InitializeProcThreadAttributeList(attList, 1, 0, &size))
            break;

        if (!::UpdateProcThreadAttribute(attList, 0, PROC_THREAD_ATTRIBUTE_PARENT\
NT_PROCESS,
            &hParent, sizeof(hParent), nullptr, nullptr))
            break;

        STARTUPINFOEX si = { sizeof(si) };
        si.lpAttributeList = attList;
        if (!::CreateProcess(nullptr, name, nullptr, nullptr, FALSE,
            EXTENDED_STARTUPINFO_PRESENT, nullptr, nullptr, (STARTUPINFO*)&si, \
&pi))
            break;
    }
}

```

```
        ::CloseHandle(pi.hProcess);
        ::CloseHandle(pi.hThread);
    } while (false);

    ::CloseHandle(hParent);
    if (attList) {
        ::DeleteProcThreadAttributeList(attList);
        ::free(attList);
    }
    return pi.dwProcessId;
}
```

Most of the code is the same as the steps outlined for working with attributes, with error handling added and proper cleanup. Notice the process handle is opened with the `PROCESS_CREATE_PROCESS` access mask. This is required when using the `PROC_THREAD_ATTRIBUTE_PARENT_PROCESS` attribute. This means not all processes can just arbitrarily serve as parents.

Running this function from within *Notepad* and a PID of an *Explorer* process created *Notepad* as expected. Opening its properties in *Process Explorer* shows *Explorer* as being the parent (figure 3-18).

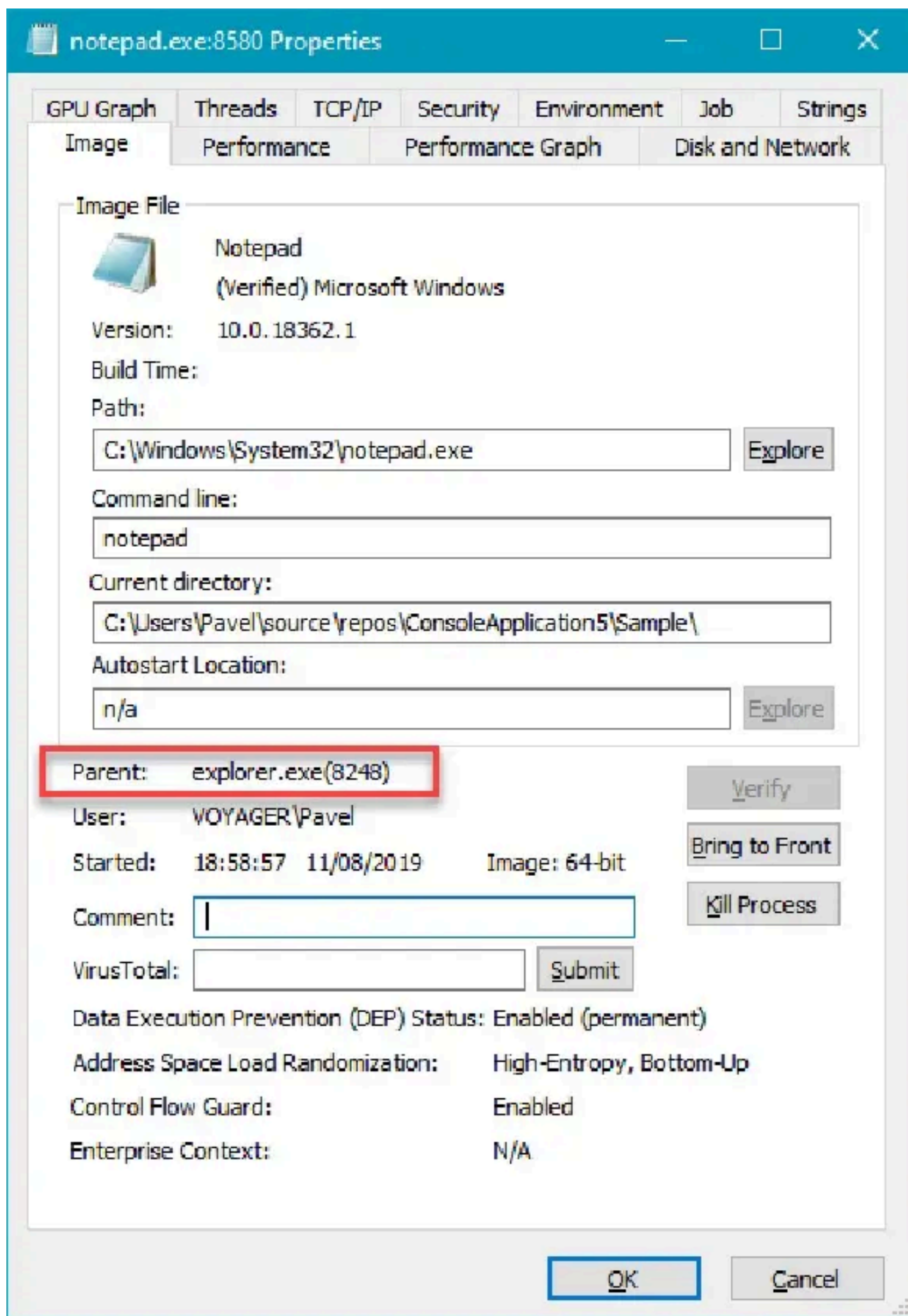


Figure 3-18: Changing parent process

As another example, consider the following function that applied a process security mitigation policy:

```

DWORD CreateProcessWithMitigations(PWSTR name, DWORD64 mitigation) {
    PROCESS_INFORMATION pi = { 0 };
    PPROC_THREAD_ATTRIBUTE_LIST attList = nullptr;

    do {
        SIZE_T size = 0;
        ::InitializeProcThreadAttributeList(nullptr, 1, 0, &size);
        if (size == 0)
            break;

        attList = (PPROC_THREAD_ATTRIBUTE_LIST)malloc(size);
        if (!attList)
            break;

        if (!::InitializeProcThreadAttributeList(attList, 1, 0, &size))
            break;

        if (!::UpdateProcThreadAttribute(attList, 0, PROC_THREAD_ATTRIBUTE_MITIGATION_POLICY,
            &mitigation, sizeof(mitigation), nullptr, nullptr))
            break;

        STARTUPINFOEX si = { sizeof(si) };
        si.lpAttributeList = attList;
        if (!::CreateProcess(nullptr, name, nullptr, nullptr, FALSE,
            EXTENDED_STARTUPINFO_PRESENT, nullptr, nullptr, (STARTUPINFO*)&si, \
            &pi))
            break;

        ::CloseHandle(pi.hProcess);
        ::CloseHandle(pi.hThread);
    } while (false);

    if (attList) {
        ::DeleteProcThreadAttributeList(attList);
        ::free(attList);
    }
    return pi.dwProcessId;
}

```

The code is nearly identical except for the different attribute. The value associated with `PROC_THREAD_ATTRIBUTE_MITIGATION_POLICY` is a `DWORD` or `DWORD64` indicating the mitigation(s) to apply to the new process.

Full discussion of process mitigations is in chapter 16.

For example, calling this function with the following arguments

```
WCHAR name[] = L"notepad";  
auto pid = CreateProcessWithMitigations(name,  
    PROCESS_CREATION_MITIGATION_POLICY_WIN32K_SYSTEM_CALL_DISABLE_ALWAYS_ON);
```

Causes *Notepad* to fail initialization and terminate, not before showing the message box in figure 3-19. The reason is that the specific mitigation prevents calls to *Win32k.sys* (the windowing manager), which essentially means *User32.dll* cannot properly initialize. Without this capability *Notepad* is useless and cannot properly execute. This mitigation is good for processes that have no UI and want to make sure *Win32k.sys* security vulnerabilities cannot be used in such a process.

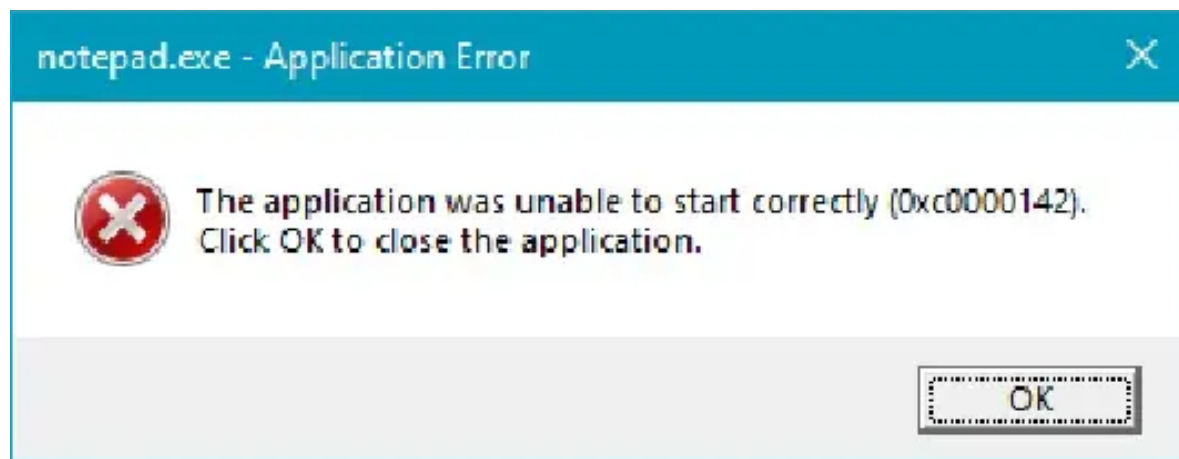


Figure 3-19: Notepad failing to initialize

Protected and PPL Processes

Protected processes were introduced in Windows Vista as a way to fight *Digital Rights Management* (DRM) infringements. These processes be granted certain access rights even by admin-level users. The only access masks allowed for protected processes are: `PROCESS_QUERY_LIMITED_INFORMATION`, `PROCESS_SET_LIMITED_INFORMATION`, `PROCESS_SUSPEND_RESUME` and `PROCESS_TERMINATE`.

Only Microsoft-signed executables with a certain *Extended Key Usage* (EKU) were allowed to execute protected.

Windows 8.1 introduced *Protected Processes Light* (PPL), which extends the protection model to include several levels of protection, where higher level protected processes have full access to lower level ones, but not vice versa. With the extended model, it is now possible to run third party anti-malware services by negotiating with Microsoft and obtaining the proper signature. Also, some of the PPL levels (such as for anti-malware) deny `PROCESS_TERMINATE` access, so that malicious software, even with elevated permissions, cannot stop or kill these services. Table 3-10 lists the PPL signer levels with a brief description.

Table 3-10: PPL signers

PPL Signer	Level	Description
WinSystem	7	System and minimal processes
WinTcb	6	Critical Windows components. <code>PROCESS_TERMINATE</code> is denied
Windows	5	Important Windows components handling sensitive data
Lsa	4	Lsass.exe (if configured to run protected)
Antimalware	3	Anti-malware service processes, including 3rd party. <code>PROCESS_TERMINATE</code> is denied
CodeGen	2	.NET native code generation
Authenticode	1	Hosting DRM content
None	0	Not valid (no protection)

The level (and whether the process is “normal” protected or PPL) shown in table 3-10 is stored inside the kernel process object.

The “limited” access masks allowed for protected/PPL processes cater for setting/querying superficial information about the process, such as querying its start time, its priority class or its executable path. Getting a list of loaded modules inside a protected process cannot be obtained because this requires an access mask of `PROCESS_QUERY_INFORMATION`, which is not allowed. Figure 3-20 shows *Csrss.exe* selected in *Process Explorer*. Notice that the list of modules in the bottom pane is empty. Also, the *Protection* column is shown, with the signer values from table 3-10.

Process	PID	CPU	Private Bytes	Working Set	Protection	Description
csrss.exe	1092	< 0.01	2,200 K	3,828 K	PeProtectedSignerWinTab-Light	Client Server Runtime Process
csrss.exe	1300	0.05	4,452 K	5,232 K	PeProtectedSignerWinTab-Light	Client Server Runtime Process
svchost.exe	11612	0.01	3,972 K	8,312 K	PeProtectedSignerWindows-Light	Host Process for Windows Services
svchost.exe	13832	0.01	12,428 K	15,784 K	PeProtectedSignerWindows-Light	Host Process for Windows Services
svchost.exe	54064		3,744 K	8,468 K	PeProtectedSignerWindows-Light	Host Process for Windows Services
SecurityHealthService.exe	3744		3,932 K	10,432 K	PeProtectedSignerWindows-Light	Windows Security Health Service
NisSrv.exe	9676		12,560 K	12,356 K	PeProtectedSignerAntimalware-Light	Microsoft Network Realtime Inspection Ser...
MsMpEng.exe	5696	0.24	406,160 K	218,740 K	PeProtectedSignerAntimalware-Light	Antimalware Service Executable
YoursPhone.exe	80564	Suspend...	33,728 K	7,104 K		
WUDFHost.exe						

Figure 3-20: Protected Processes

Figure 3-20 also shows Microsoft's own anti-malware executables (*MsMpEng.exe* and *NisSrv.exe*, known as "Windows Defender") running as anti-malware PPL, just like other 3rd party anti-malware services.

Protected and PPL processes cannot load arbitrary DLLs, so that the protected process is not tricked into loading an untrusted DLL that would run under the protection of the process. All DLLs loaded by protected/PPL processes must be signed properly.

Creating a process as protected requires the `CREATE_PROTECTED_PROCESS` flag in `CreateProcess`. Of course, this can only work on properly signed executables. The protection mechanism is too specialized to be used by normal applications and so is not going to be discussed further in this book.



You can find more information on protected/PPL processes in the "Windows Internals 7th edition Part 1" book in chapter 3.

UWP Processes

A *Universal Windows Platform* (UWP) process is similar to any other standard process. It uses the Windows Runtime platform/API to do most of its work - be that UI, graphics, networking, background processing and so on. Some of its unique properties include:

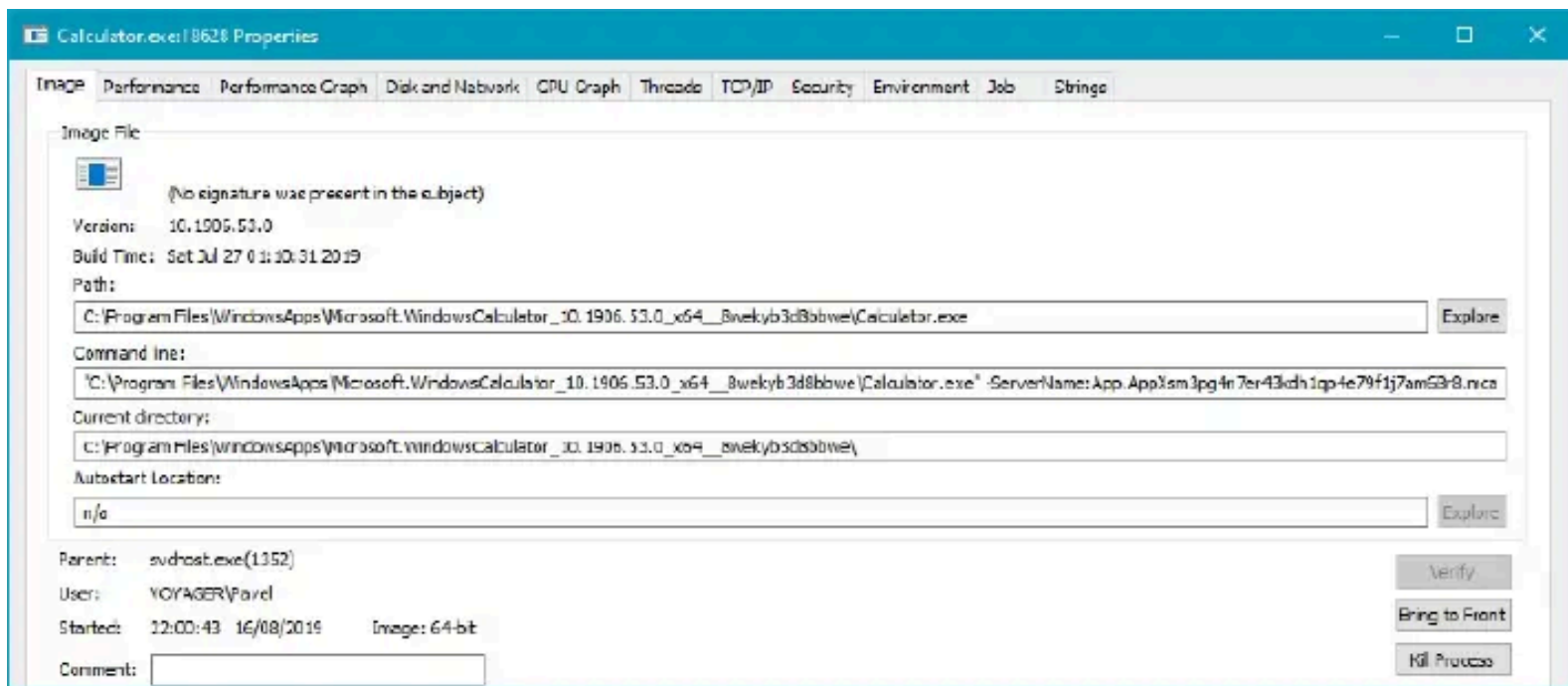
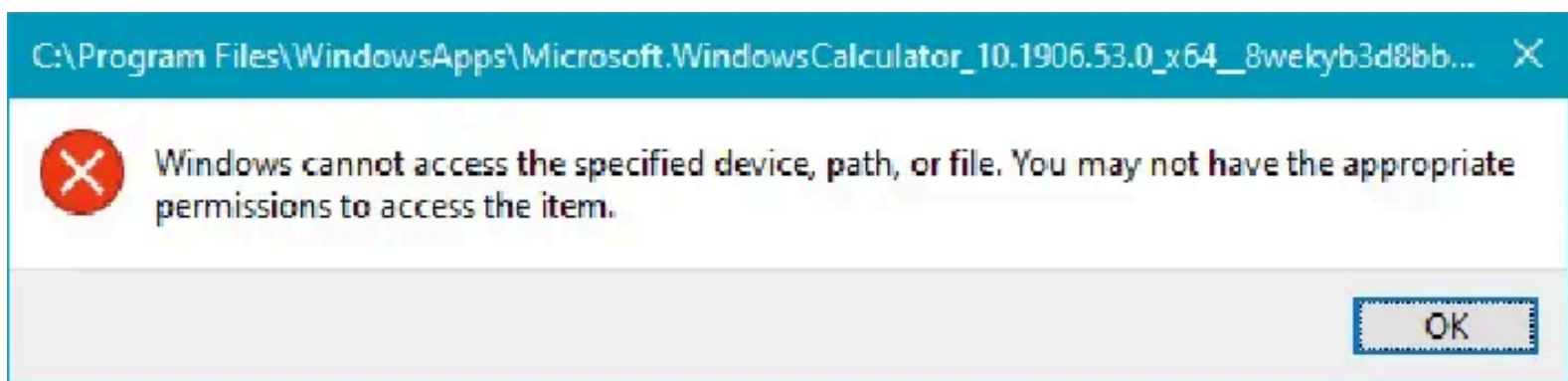
- A UWP process always runs under an application sandbox known as *AppContainer* that limits what it can do and what it can access (discussed in more detail in chapter 16, “Security”).
- A UWP process’ state is managed by the *Process Lifetime Manager* (PLM), running under the *Explorer.exe* process, that can initiate process suspension, resumption, and termination based on its foreground/background activity and memory usage (discussed more in chapter 18).
- A UWP package includes a set of capabilities - declarations - of what the application wants to access (such as camera, location, Pictures folder) and those capabilities are listed in the Microsoft Store so that users can decide whether that would like to download such an application.
- UWP processes are single instance by default (multiple instances supported starting with Windows 10 version 1803).

From a process creation perspective, a standard `CreateProcess` call cannot create a UWP process. This is because a UWP application has **identity**, something that is missing from standard executables. Such an application is built into a package with the executable, libraries, resource files and anything else that is needed for the application to execute properly. This package has a universally unique name, and this name is the one required for UWP process creation.



You can view this name in *Task Manager* or *Process Explorer* by adding the *Package Name* column.

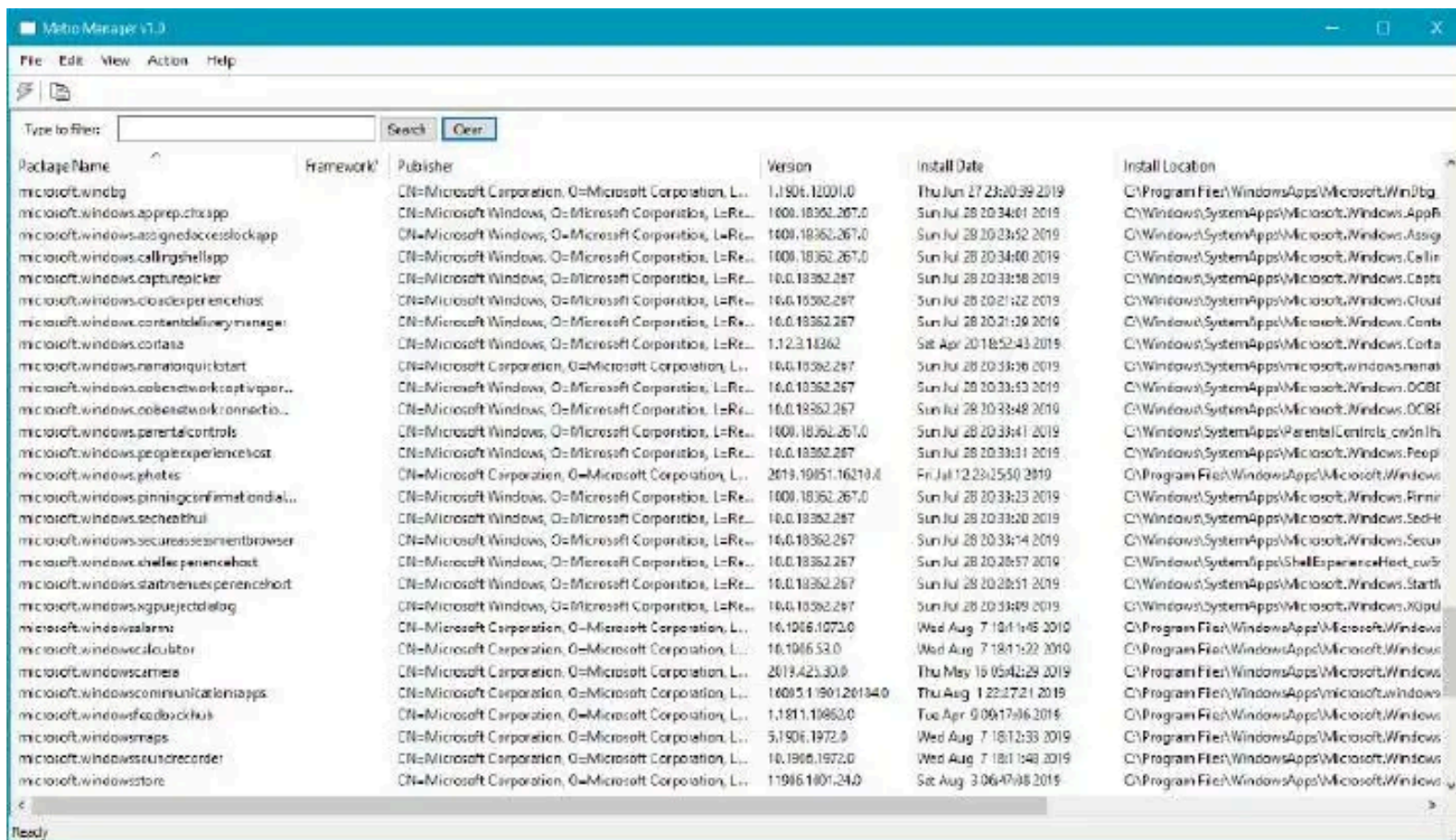
As a simple example to this requirement, run *Calculator* on Windows 10, and look at its properties with *Process Explorer* (figure 3-21). Notice the command line; disregard its length and ugliness and just copy it, and use *Start/Run* to launch another calculator with the pasted in the command-line. You’ll get an error message box similar to the one in figure 3-22.

Figure 3-21: *Calculator* properties in *Process Explorer*Figure 3-22: Error message box when creating *Calculator* manually

The error message in figure 3-22 seems unrelated to anything. The missing piece of information is the package full name, which needs to be specified as a process attribute. Unfortunately, this particular attribute is undocumented, so cannot be specified with the help of the Windows headers. There is a way to specify this parameter using another creation mechanism devised just for this purpose and exposed through a COM interface (and class).

The *MetroManager* application, shown in figure 3-23 lists the available UWP packages on the machine, and allows the user to launch any selected package. The application demonstrates a few interesting abilities:

- Consuming Windows Runtime APIs from a non-UWP application.
- Enumerating packages
- Launching UWP processes in the documented way.



The screenshot shows the Metro Manager v1.0 application window. It has a menu bar with File, Edit, View, Action, and Help. Below the menu bar is a search bar with a 'Search' button and a 'Clear' button. The main area displays a table of installed apps with the following columns: Package Name, Framework, Publisher, Version, Install Date, and Install Location. The table lists various Microsoft Windows apps, including windows, windows.app, windows.assignedtasks, windows.callingshell, windows.capturepicker, windows.cloudexperiencehost, windows.contentdeliverymanager, windows.cortana, windows.namotiquistart, windows.cobesetworkcaptivaper..., windows.cobesetworkconnection..., windows.parentalcontrols, windows.peopleexperiencehost, windows.photocenter, windows.pinningsconfirmationdial..., windows.sechelp, windows.secureassessmentbrowser, windows.challengeexperiencehost, windows.startmenuexperiencehost, windows.xgputextdiag, windows.walrus, windows.walruscalculator, windows.walruscamera, windows.walruscommunicationapps, windows.walrusfeedbackhub, windows.walrusmaps, windows.walrusmusicrecorder, and windows.walrusstore.

Package Name	Framework	Publisher	Version	Install Date	Install Location
microsoft.windows		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	1.1906.12001.0	Thu Jun 27 23:20:39 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.app		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	1000.18362.267.0	Sun Jul 28 20:34:01 2019	C:\Windows\SystemApps\Microsoft\Windows.AppR...
microsoft.windows.assignedtasks		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	1000.18362.267.0	Sun Jul 28 20:33:52 2019	C:\Windows\SystemApps\Microsoft\Windows.Assign...
microsoft.windows.callingshell		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	1000.18362.267.0	Sun Jul 28 20:34:00 2019	C:\Windows\SystemApps\Microsoft\Windows.Callin...
microsoft.windows.capturepicker		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:33:58 2019	C:\Windows\SystemApps\Microsoft\Windows.Captu...
microsoft.windows.cloudexperiencehost		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:21:22 2019	C:\Windows\SystemApps\Microsoft\Windows.Cloud...
microsoft.windows.contentdeliverymanager		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:21:39 2019	C:\Windows\SystemApps\Microsoft\Windows.Conte...
microsoft.windows.cortana		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	1.12.3.18362	Sat Apr 20 18:52:43 2019	C:\Windows\SystemApps\Microsoft\Windows.Corti...
microsoft.windows.namotiquistart		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	10.0.18362.267	Sun Jul 28 20:33:36 2019	C:\Windows\SystemApps\Microsoft\Windows.namot...
microsoft.windows.cobesetworkcaptivaper...		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:33:53 2019	C:\Windows\SystemApps\Microsoft\Windows.COB...
microsoft.windows.cobesetworkconnection...		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:33:48 2019	C:\Windows\SystemApps\Microsoft\Windows.COB...
microsoft.windows.parentalcontrols		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	1000.18362.267.0	Sun Jul 28 20:33:41 2019	C:\Windows\SystemApps\ParentalControls\parental...
microsoft.windows.peopleexperiencehost		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:33:31 2019	C:\Windows\SystemApps\Microsoft\Windows.Peop...
microsoft.windows.photocenter		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	2019.19051.16219.0	Fri Jul 12 23:25:50 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.pinningsconfirmationdial...		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	1000.18362.267.0	Sun Jul 28 20:33:23 2019	C:\Windows\SystemApps\Microsoft\Windows.Pinn...
microsoft.windows.sechelp		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:33:20 2019	C:\Windows\SystemApps\Microsoft\Windows.Sech...
microsoft.windows.secureassessmentbrowser		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:33:14 2019	C:\Windows\SystemApps\Microsoft\Windows.Secu...
microsoft.windows.challengeexperiencehost		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:28:57 2019	C:\Windows\SystemApps\ShellExperienceHost\cu...
microsoft.windows.startmenuexperiencehost		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:28:51 2019	C:\Windows\SystemApps\Microsoft\Windows.Start...
microsoft.windows.xgputextdiag		CN=Microsoft Windows, O=Microsoft Corporation, L=Re...	10.0.18362.267	Sun Jul 28 20:33:09 2019	C:\Windows\SystemApps\Microsoft\Windows.XGpu...
microsoft.windows.walrus		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	10.1006.1072.0	Wed Aug 7 18:11:46 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.walruscalculator		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	10.1006.53.0	Wed Aug 7 18:11:22 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.walruscamera		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	2019.423.30.0	Thu May 16 05:42:29 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.walruscommunicationapps		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	1000.5.1.1901.20134.0	Thu Aug 1 22:27:21 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.walrusfeedbackhub		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	1.1811.19062.0	Tue Apr 9 09:17:46 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.walrusmaps		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	5.1906.1972.0	Wed Aug 7 18:12:33 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.walrusmusicrecorder		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	10.1906.1972.0	Wed Aug 7 18:11:48 2019	C:\Program Files\WindowsApps\Microsoft\Win...
microsoft.windows.walrusstore		CN=Microsoft Corporation, O=Microsoft Corporation, L=...	11906.1901.24.0	Sat Aug 3 06:47:08 2019	C:\Program Files\WindowsApps\Microsoft\Win...

Figure 3-23: The Metro Manager application

The Windows Runtime (WinRT) is built on top of COM, which means it utilizes interfaces, classes, class factories, GUIDs and other concepts from the COM world (albeit giving to some new names). The Windows Runtime supports entities typically found in high level languages, including static methods and generics. We'll look at how this works in chapter 18. Here, I want to concentrate mostly on the mechanics of using the Windows Runtime.

The Windows Runtime API can be consumed by a C++ client in several ways:

1. Directly, by instantiating the proper classes and working with the low level object factories until an instance is created and then use normal COM calls.
2. Use the *Windows Runtime Library* (WRL) C++ wrappers and helpers.
3. Use the C++/CX language extensions, that provide an easy access to WinRT by extending C++ in a non-standard way.
4. Use the CppWinRT library, that provides relatively easy access to the WinRT APIs with standard C++ only.

All the above four ways are officially supported. The first option is the most tedious and is only recommended for learning purposes, as it hides very little from the developer and so is very verbose. Option 2 is easier to use, but is not favored today, not even by Microsoft. Option 3 is the easiest and was the most common in the early days of WinRT, but is frowned

upon today because it forces the developer to use non-standard C++ extensions. This leaves option 4, which is the recommended way to work with WinRT in C++, as it's easy enough to work with but still uses standard C++ constructs.

CppWinRT is beyond the scope of this book, but we'll cover the basics which can get you quite far. First, we need to add the Nuget package for the library (figure 3-24). Next, we need to add includes for the namespaces that we wish to use from WinRT. The following was added to the pre-compiled header (*pch.h* in the project source code):

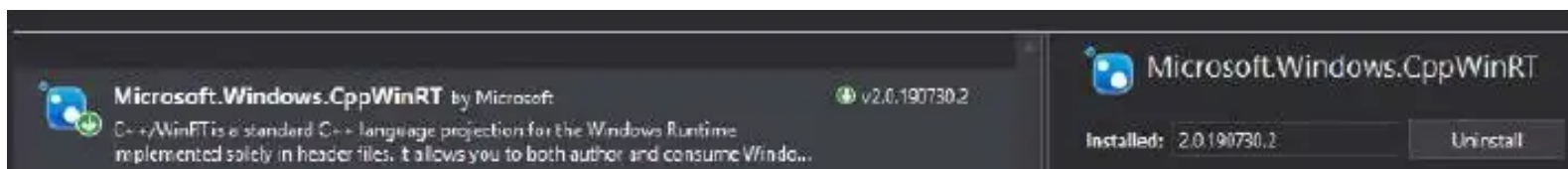


Figure 3-24: CppWinRT Nuget package

More CppWinRT information is available in the online Microsoft documentation.

```
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.ApplicationModel.h>
#include <winrt/Windows.Management.Deployment.h>
#include <winrt/Windows.Storage.h>
```

This is the general format of the CppWinRT headers: `winrt` prefix and then the namespace (found in the WinRT documentation).

The headers without the `winrt` prefix are the “real” WinRT headers, that are included internally. You generally don't want these headers when working with CppWinRT.

Since all WinRT APIs are in namespaces and nested namespaces, the type names become long. It's much easier to add using namespace statements to a source file, or in some cases - a function to ease accessing the various types:

```
using namespace winrt;
using namespace winrt::Windows::Management::Deployment;
using namespace winrt::Windows::ApplicationModel;
```

The `winrt` namespace itself has some general CppWinRT helpers, so it's mandatory. The others depend on the types we use.

Enumerating the UWP packages is done with the `PackageManager` class in the `winrt::Windows::Management::Deployment` namespace:

```
auto packages = PackageManager().FindPackagesForUser(L "");
```

An empty string for the user looks at the current user (other users on the system may have installed different packages).

The `auto` keyword is real helper here as the actual returned type is `IIterable<Package>` (and I've shortened both types assuming we have using namespace for them). The important point is this method returns a collection (`IIterable<>`) in WinRT parlance. Because of the added conveniences in CppWinRT, any such collection can be iterated upon with the C++ enhanced range-based for statement:

```
for (auto package : packages) {
    auto item = std::make_shared<AppItem>();
    item->InstalledLocation = package.InstalledLocation().Path().c_str();
    item->FullName = package.Id().FullName().c_str();
    item->InstalledDate = package.InstalledDate();
    item->IsFramework = package.IsFramework();
    item->Name = package.Id().Name().c_str();
    item->Publisher = package.Id().Publisher().c_str();
    item->Version = package.Id().Version();

    m_AllPackages.push_back(item);
}
```

`AppItem` is a normal C++ class the application defines to store the information in plain C++ types rather than WinRT types where it makes sense, especially for strings:

```

struct AppItem {
    CString Name, Publisher, InstalledLocation, FullName;
    winrt::Windows::ApplicationModel::PackageVersion Version;
    winrt::Windows::Foundation::DateTime InstalledDate;
    bool IsFramework;
};

```



The standard Windows Runtime string is typed as `HSTRING`, which is an immutable array of UTF-16 characters stored with its length.

This `AppItem`-stored data is used to show the information in the application's list view.

Running a UWP package is accomplished with the following COM (not WinRT) interface, shown in C++:

```

struct IApplicationActivationManager : public IUnknown {
    virtual HRESULT __stdcall ActivateApplication(
        /* [in] */ LPCWSTR appUserModelId,
        /* [unique][in] */ LPCWSTR arguments,
        /* [in] */ ACTIVATION_OPTIONS options,
        /* [out] */ DWORD *processId) = 0;

    virtual HRESULT __stdcall ActivateForFile(
        /* [in] */ LPCWSTR appUserModelId,
        /* [in] */ IShellItemArray *itemArray,
        /* [unique][in] */ LPCWSTR verb,
        /* [out] */ DWORD *processId) = 0;

    virtual HRESULT __stdcall ActivateForProtocol(
        /* [in] */ LPCWSTR appUserModelId,
        /* [in] */ IShellItemArray *itemArray,
        /* [out] */ DWORD *processId) = 0;
};

```

There are several ways to “activate” a UWP application, using something called *contracts*, where one of the contracts is called *Launch*, which naturally launches the application. `ActivateApplication` is using the *Launch* contract, while the others work with different contracts. We'll use the *Launch* contract only in this application.

The complete code for launching an app is in the `CView::RunApp` member function. First, we need to locate some information about a package using its unique package full name (stored in the `AppItem` structure shown earlier). Here is the first call:

```
bool CView::RunApp(PCWSTR fullPackageName) {
    PACKAGE_INFO_REFERENCE pir;
    int error = ::OpenPackageInfoByFullName(fullPackageName, 0, &pir);
    if (error != ERROR_SUCCESS)
        return false;
```

`OpenPackageInfoByFullName` returns an opaque pointer to an internal data structure that holds information about the requested package. Unfortunately, the full package name is not enough, because theoretically a package can contain multiple applications (which is not yet supported) and so another application ID needs to be extracted from the package:

```
UINT32 len = 0;
error = ::GetPackageApplicationIds(pir, &len, nullptr, nullptr);
if (error != ERROR_INSUFFICIENT_BUFFER)
    break;

auto buffer = std::make_unique<BYTE[]>(len);
UINT32 count;
error = ::GetPackageApplicationIds(pir, &len, buffer.get(), &count);
if (error != ERROR_SUCCESS)
    break;
```

This is accomplished in two steps: first calling `GetPackageApplicationIds` with a `NULL` pointer for the application ID and a length of zero. This causes the function to fill in the required length. Then, a buffer is constructed with `make_unique` (ensuring it's automatically destroyed when the variable goes out of scope), and a second call is made.

The application IDs returned are stored with a 4-byte length and then the data itself. Since only one application is expected, we can just skip the first 4 bytes and use the rest as the application ID. The last step is to create the instance that implements `IApplicationActivationManager` and call `ActivateApplication` with the application ID:

```
CComPtr<IApplicationActivationManager> mgr;  
auto hr = mgr.CoCreateInstance(CLSID_ApplicationActivationManager);  
if (FAILED(hr))  
    break;  
  
DWORD pid;  
hr = mgr->ActivateApplication((PCWSTR)(buffer.get() + sizeof(ULONG_PTR)),  
    nullptr, AO_NOERRORUI, &pid);
```

ActivateApplication is even kind enough to return the process ID of the created process. Finally, the package information data needs to be freed:

```
::ClosePackageInfo(pir);
```

If you look at the parent of any UWP process, you'll discover it's an *Svchost.exe* process, rather than the direct creator (see figure 3-22). This is because UWP processes are actually launched by the *DCOM Launch* service, hosted in a service host (figure 3-25).

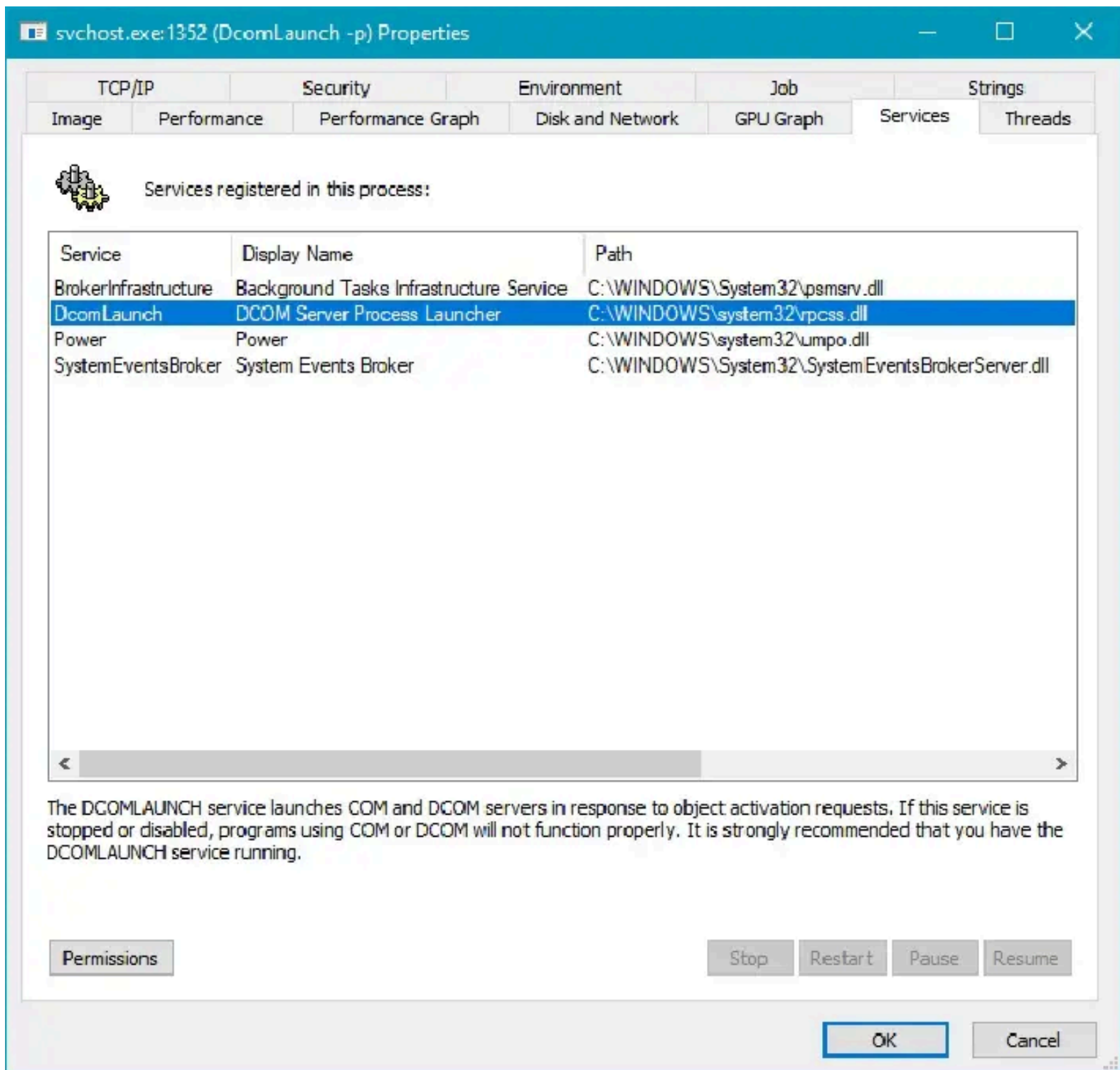


Figure 3-25: The DCOM Launch service

Minimal and Pico Processes

Minimal processes contain just a user mode address space. *Memory Compression* and *Registry* are canonical examples of minimal processes. Minimal processes can only be created by the kernel, and so will not be discussed further in this book.

Pico processes are minimal processes with an added twist: a pico provider, which is a kernel driver that is responsible for translating Linux system calls to equivalent Windows system

calls. This is the basis of the *Windows Subsystem for Linux* (WSL), available in Windows 10 version 1607 (and later) and Windows 2016 (and later). Pico processes are beyond the scope of this book, although I may publish a special chapter on WSL in the future.

Process Termination

Most processes will terminate at some point before the system is shutdown. There are several ways a process might exit or terminate. One thing to keep in mind is no matter how a process terminates, the kernel ensures nothing private to the process remains: all private (non-shared) memory is freed and all handles in the process handle table are closed.

A process terminates if any one of the following conditions is met:

1. All the threads in the process exit or terminate.
2. Any thread in the process calls `ExitProcess`.
3. The process is terminated (usually externally but could be because of an unhandled exception) with `TerminateProcess`.

Anyone writing a Windows application usually finds out at some point that the thread executing the main function is “special”, typically referred to as the *main thread*. It can be observed that whenever the main function returns, the process exits. This seems to be a scenario not listed in the above reasons for process exit. However, it does, and it’s scenario number 2. The C/C++ runtime library invokes `main/WinMain` (discussed in the section “Process Creation” earlier in this chapter), and then does required cleanup such as calling global C++ destructors, C runtime cleanup, etc., and then as its final act eventually calls `ExitProcess`, causing the process to exit.

From the kernel’s perspective, all threads in a process are equal, and there is no *main* thread. The kernel destroys a process when all threads within it exit/terminate, as a process without threads is mostly useless. In practice, this scenario can only be achieved in native processes (executables that only depend on *NtDll.dll* and have no C/C++ runtime). In other words, this is unlikely to happen in normal Windows programming.

The `ExitProcess` function is defined like so:

```
void ExitProcess(_In_ UINT exitCode);
```

The calling process is the only one capable of calling `ExitProcess`, and naturally, this function never returns. External processes can attempt to terminate a process with `TerminateProcess` (discussed later). The exit code becomes the process exit code, that can be read by anyone holding a handle to the process with `GetExitCodeProcess` defined like so:

```
BOOL GetExitCodeProcess(  
    _In_ HANDLE hProcess,  
    _Out_ LPDWORD lpExitCode);
```

It might seem weird that the exit code is available **after** the process exits, but since a handle to the process is still open, the kernel process structure is still alive and that is where the exit code is stored. What would happen if `GetExitCodeProcess` is called for a live process. You might expect the function to fail, but confusingly it succeeds and returns an exit code called `STILL_ACTIVE` (0x103).



In the kernel, `STILL_ACTIVE` is called `STATUS_PENDING`, indicating in this case that the process is still alive.

The last sentence means looking at the exit code is not a 100% sure way to check if a process is still alive. The proper way to do this is call `WaitForSingleObject(hProcess, 0)` and check the return value against `WAIT_OBJECT_0`; if equal, the process is dead; Only the kernel management object still remains because there is at least one open handle to the process.

`ExitProcess` shuts down the process in an orderly fashion, performing the following important actions:

1. All other threads in the process terminate.
2. All DLLs in the process get their `DllMain` function called with a reason value of `PROCESS_DLL_DETACH`, indicating the DLL is about to be unloaded and should do its cleanup.
3. Terminates the process and the calling thread (`ExitProcess` never returns).

The third way a process might terminate is because of a call to `TerminateProcess`, defined like so:

```
BOOL TerminateProcess(  
    _In_ HANDLE hProcess,  
    _In_ UINT    uExitCode);
```

`TerminateProcess` can be called from outside the process, assuming a handle with the access mask `PROCESS_TERMINATE` can be obtained. The function terminates the process then and there and specifies what its exit value would be. The process in question has no say in this.

`TerminateProcess` differs from `ExitProcess` in one important aspect: `DllMain` functions for all DLLs in the target process don't get called, so cannot perform any cleanup action. This may result in loss of functionality or data. For example, if a DLL writes some information to a log file when it's unloaded, it will not get the chance to do so. Clearly, `TerminateProcess` should be used as a last resort.



Task Manager's End Task button in the *Details* tab calls `TerminateProcess` if it's able to open a process handle with `PROCESS_TERMINATE` access mask. The *End Task* button in the *Processes* tab is trickier, as for GUI processes it first attempts to ask the process to exit nicely by sending a close message to its main window (`SendMessage` or `PostMessage` function with `WM_CLOSE` as the message type).

Enumerating Processes

In some cases it's beneficial to enumerate existing processes. One possible reason would be to look for a particular process of interest. Another reason may be to create some sort of tool that provides information on existing processes. Well-known tools such as *Task Manager* and *Process Explorer* use process enumeration.

The Windows API provides three documented ways to enumerate processes. We'll examine all of them and then briefly discuss a fourth, semi-documented, option.

Using EnumProcesses

The simplest function to use is `EnumProcesses` part of the so-called *Process Status API* (PSAPI), included in `<psapi.h>`:

```

BOOL EnumProcesses(
    _Out_ DWORD *pProcessIds,
    _In_  DWORD cb,
    _Out_ DWORD *pBytesReturned);

```

This function provides the bare minimum - all process IDs. The caller must allocate a large enough buffer to store all PIDs. On return, the function indicates the number of bytes actually stored in the provided buffer. If it's lower than the buffer size, it means the buffer was large enough to contain all PIDs. If it's equal to the buffer size, then the buffer was probably too small, and the caller should make a second call with a larger buffer size until the first condition is met.

The simplest way is to call the function with a (hopefully) large enough buffer:

```

const int MaxCount = 1024;
DWORD pids[MaxCount];
DWORD actualSize;
if(::EnumProcesses(pids, sizeof(pids), &actualSize)) {
    // assume actualSize < sizeof(pids)

    int count = actualSize / sizeof(DWORD);
    for(int i = 0; i < count; i++) {
        // do something with pids[i]
    }
}

```

A more conservative approach needs to allocate the PIDs array dynamically so it can be resized when needed. A relatively simple way to do this is by leveraging the C++ `std::unique_ptr<>` template class. Here is a revised example:

```
int maxCount = 256;
std::unique_ptr<DWORD[]> pids;
int count = 0;

for (;;) {
    pids = std::make_unique<DWORD[]>(maxCount);
    DWORD actualSize;
    if (!::EnumProcesses(pids.get(), maxCount * sizeof(DWORD), &actualSize))
        break;

    count = actualSize / sizeof(DWORD);
    if (count < maxCount)
        break;

    // need to resize
    maxCount *= 2;
}

for (int i = 0; i < count; i++) {
    // do something with pids[i]
}
```



Remember to add `#include <psapi.h>` for the above to compile.

You'll need to `#include <memory>` so that `unique_ptr<>` is available.

If you would rather use classic C++ or even C, you can of course do that by leveraging the `new` and `delete` operators or the `malloc` and `free` functions. I recommend using the modern C++ approach which is less error prone because the allocated memory is freed automatically when the relevant object is destroyed.

The downside of `EnumProcesses` is its minimal information - just the PID for each process. If anything else about the process is needed (the usual case), another call to `OpenProcess` is required to get a handle to each process of interest and then making the appropriate

calls to retrieve information or perform the required operations on the process. Of course, `OpenProcess` can fail, because not every access mask is necessarily possible to obtain for a process.

The following code snippet shows how to get the process image name and its start time after a successful call to `EnumProcesses`:

```
// count is the number of processes
for (int i = 0; i < count; i++) {
    DWORD pid = pids[i];
    HANDLE hProcess = ::OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION, FALSE, p\
id);
    if (!hProcess) {
        printf("Failed to open a handle to process %d (error=%d)\n",
            pid, ::GetLastError());
        continue;
    }

    FILETIME start = { 0 }, dummy;
    ::GetProcessTimes(hProcess, &start, &dummy, &dummy, &dummy);
    SYSTEMTIME st;
    ::FileTimeToLocalFileTime(&start, &start);
    ::FileTimeToSystemTime(&start, &st);

    WCHAR exeName[MAX_PATH];
    DWORD size = MAX_PATH;
    DWORD count = ::QueryFullProcessImageName(hProcess, 0, exeName, &size);
    printf("PID: %5d, Start: %d/%d/%d %02d:%02d:%02d Image: %ws\n",
        pid, st.wDay, st.wMonth, st.wYear, st.wHour, st.wMinute, st.wSecond,
        count > 0 ? exeName : L"Unknown");
}
```

`OpenProcess` is used to get a handle to the process with `PROCESS_QUERY_LIMITED_INFORMATION` access mask, which is the lowest one you can ask for. This is enough to get shallow information on a process, such as its start time (`GetProcessTimes`) or image file name (`QueryFullProcessImageName`).

Here is the prototype for `GetProcessTimes`:

```
BOOL GetProcessTimes(  
    _In_ HANDLE hProcess,  
    _Out_ LPFILETIME lpCreationTime,  
    _Out_ LPFILETIME lpExitTime,  
    _Out_ LPFILETIME lpKernelTime,  
    _Out_ LPFILETIME lpUserTime);
```

FILETIME is a 64-bit value that is split into two 32-bit values. The creation and exit time are given in 100 nano-seconds units measured from January 1, 1601, UTC, at midnight. The creation time is converted into a more manageable form using `FileTimeToSystemTime`, which splits the 64-bit value into human-readable pieces (day, month, year, etc.).

The kernel and user times are relative, given in the same 100 nsec units. They are not used in the above code, putting their result in a dummy variable. Note that providing NULL for any of the parameters will cause the function to throw an access violation exception.

`QueryFullProcessImageName` allows getting the full executable path of a given process:

```
BOOL QueryFullProcessImageName(  
    _In_     HANDLE hProcess,  
    _In_     DWORD  dwFlags,  
    _Out_    LPTSTR lpExeName,  
    _Inout_  PDWORD lpdwSize  
);
```

The `dwFlags` parameter is usually zero, but can be `PROCESS_NAME_NATIVE` (1), which returns the path in device form, which is the native Windows way of representing paths (something like `\Device\HarddiskVolume3\MyDir\MyApp.exe`). We'll look at this form more closely in chapter 11. The `lpExeName` parameter is the buffer allocated by the caller, and the last parameter is a pointer to the size of the buffer, in characters. This is an input/output parameter, so it must be initialized to the allocated buffer size, and the function changes it to the actual number of characters written to the buffer.

Running this code with standard user rights produces output like the following:

```

Failed to get a handle to process 0 (error=87)
Failed to get a handle to process 4 (error=5)
Failed to get a handle to process 88 (error=5)
Failed to get a handle to process 152 (error=5)
Failed to get a handle to process 900 (error=5)
Failed to get a handle to process 956 (error=5)
Failed to get a handle to process 1212 (error=5)
...
PID: 9796, Start: 26/7/2019 14:13:40 Image: C:\Windows\System32\sihost.exe
PID: 9840, Start: 26/7/2019 14:13:40 Image: C:\Windows\System32\svchost.exe
Failed to get a handle to process 9864 (error=5)
PID: 9904, Start: 26/7/2019 14:13:40 Image: C:\Windows\System32\svchost.exe
PID: 9936, Start: 26/7/2019 14:13:40 Image: C:\Windows\System32\svchost.exe
PID: 10004, Start: 26/7/2019 14:13:40 Image: C:\Windows\System32\taskhostw.exe
Failed to get a handle to process 10032 (error=5)
PID: 9556, Start: 26/7/2019 14:13:40 Image: C:\Windows\explorer.exe
...

```

Getting a handle to PID 0 (*System Idle Process* in *Task Manager*) fails, as PID 0 is not valid. This is why the error is number 87 (ERROR_INVALID_PARAMETER). Other processes in the low-PID range fail to open as well, with error 5 (ERROR_ACCESS_DENIED).

Running the same code with admin rights by opening an elevated command window, navigating to the output directory and running the executable produces the following output:

```

Failed to get a handle to process 0 (error=87)
PID: 4, Start: 26/7/2019 14:13:20 Image: Unknown
PID: 88, Start: 26/7/2019 14:13:01 Image: Unknown
PID: 152, Start: 26/7/2019 14:13:01 Image: Unknown
PID: 900, Start: 26/7/2019 14:13:20 Image: C:\Windows\System32\smss.exe
Failed to get a handle to process 956 (error=5)
PID: 1212, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\wininit.exe
Failed to get a handle to process 1220 (error=5)
PID: 1288, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\services.exe
PID: 1300, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\LsaIso.exe
PID: 1316, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\lsass.exe
...

```

There are two things to notice here. First, we do get more processes open successfully than with standard user rights. Second, with some processes we fail to retrieve the image name.

This is because they don't have normal executable names. These are some of the "special" ones: 4 (*System*), 88 (*Secure System*), 152 (*Registry*) and further down (not shown) *Memory Compression*. Clearly, `QueryFullProcessImageName` cannot provide the names for these processes.

Even with administrator privileges, we can't seem to open every possible process. This situation can be improved by enabling the *Debug* privilege (which exists by default in the admin token, but is not enabled). Curiously enough, if you run the code directly from Visual Studio (running elevated), you'll get the same effect as if the *Debug* privilege was enabled, because Visual Studio already enabled its *Debug* privilege and because it's the one launching the process, its access token is duplicated for the new process, so that the *Debug* privilege is already enabled.

To make sure the *Debug* privilege is enabled regardless of where the executable is launched from, we can use the following function:

```
bool EnableDebugPrivilege() {
    wil::unique_handle hToken;
    if (!::OpenProcessToken(::GetCurrentProcess(), TOKEN_ADJUST_PRIVILEGES,
        hToken.addressof()))
        return false;

    TOKEN_PRIVILEGES tp;
    tp.PrivilegeCount = 1;
    tp.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;
    if (!::LookupPrivilegeValue(nullptr, SE_DEBUG_NAME, &tp.Privileges[0].Luid))
        return false;

    if (!::AdjustTokenPrivileges(hToken.get(), FALSE, &tp, sizeof(tp),
        nullptr, nullptr))
        return false;

    return ::GetLastError() == ERROR_SUCCESS;
}
```



The complete project is named *ProcEnum* in the samples repository.

A thorough explanation of this function is saved for chapter 16 (“Security”). For now, we can simply use it and get better results:

```
Failed to get a handle to process 0 (error=87)
PID:      4, Start: 26/7/2019 14:13:20 Image: Unknown
PID:     88, Start: 26/7/2019 14:13:01 Image: Unknown
PID:    152, Start: 26/7/2019 14:13:01 Image: Unknown
PID:    900, Start: 26/7/2019 14:13:20 Image: C:\Windows\System32\smss.exe
PID:    956, Start: 26/7/2019 14:13:30 Image: C:\Windows\System32\csrss.exe
PID:   1212, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\wininit.exe
PID:   1220, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\csrss.exe
PID:   1288, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\services.exe
PID:   1300, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\LsaIso.exe
PID:   1316, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\lsass.exe
PID:   1436, Start: 26/7/2019 14:13:33 Image: C:\Windows\System32\svchost.exe
...
```

Now we can open a handle to each and every process (except PID 0 which of course is not a real process).

We still are unable to get the names of the special processes. The next technique for process enumeration solves this issue.

Using the Toolhelp Functions

The so-called “Toolhelp” functions provide a more convenient way to get basic information on processes, including process “names” for the special processes not based on an executable image. All this is available from a standard user rights process - no need for elevated permissions.

To gain access to these functions, include `<tlhelp32.h>`. The initial function to call is `CreateToolhelp32Snapshot`, which creates a snapshot consisting of an optional combination of processes and threads, and for a specific process - heaps and modules as well. Here is the call to create the snapshot to get information for processes only:

```

HANDLE hSnapshot = ::CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
if (hSnapshot == INVALID_HANDLE_VALUE) {
    // handle error
}

```

The second parameter to `CreateToolhelp32Snapshot` indicates which process is the target of the snapshot in case modules or heaps are requested. For processes and threads this must be zero and all processes/threads are included in the snapshot.

Now process enumeration starts with `Process32First`, and subsequent processes are available by calling `Process32Next` until the latter returns `FALSE` meaning there are no more processes. Both functions accept a `PROCESSENTRY32` structure pointer where the information is returned for each process:

```

typedef struct tagPROCESSENTRY32 {
    DWORD    dwSize;                // size of structure
    DWORD    cntUsage;              // unused
    DWORD    th32ProcessID;         // PID
    ULONG_PTR th32DefaultHeapID;    // unused
    DWORD    th32ModuleID;          // unused
    DWORD    cntThreads;            // # threads
    DWORD    th32ParentProcessID;   // parent PID
    LONG     pcPriClassBase;        // Base priority
    DWORD    dwFlags;               // unused
    TCHAR    szExeFile[MAX_PATH];   // Path
} PROCESSENTRY32;

```

The first member (`dwSize`) must be set to the size of the structure. As can be seen from the comments, only some of the members are actually used. The following code demonstrates getting all possible information provided by process snapshotting:

```

PROCESSENTRY32 pe;
pe.dwSize = sizeof(pe);

if (!::Process32First(hSnapshot, &pe)) {
    // unlikely - handle error
}

do {
    printf("PID:%6d (PPID:%6d): %ws (Threads=%d) (Priority=%d)\n",
        pe.th32ProcessID, pe.th32ParentProcessID, pe.szExeFile,
        pe.cntThreads, pe.pcPriClassBase);
} while (::Process32Next(hSnapshot, &pe));

::CloseHandle(hSnapshot);

```

The project *ProcList* from this chapter's samples has the complete code. Here is the first few lines of output:

```

PID:      0 (PPID:      0): [System Process] (Threads=12) (Priority=0)
PID:      4 (PPID:      0): System (Threads=359) (Priority=8)
PID:     88 (PPID:      4): Secure System (Threads=0) (Priority=8)
PID:    152 (PPID:      4): Registry (Threads=4) (Priority=8)
PID:    900 (PPID:      4): smss.exe (Threads=2) (Priority=11)
PID:    956 (PPID:    932): csrss.exe (Threads=13) (Priority=13)
PID:   1212 (PPID:    932): wininit.exe (Threads=2) (Priority=13)
PID:   1220 (PPID:   1204): csrss.exe (Threads=27) (Priority=13)

```

Using the WTS Functions

The *Windows Terminal Services* (WTS) functions are used to work in a terminal services (also called *Remote Desktop Services*) environment, where a server may host several remote (and local) sessions simultaneously. That said, the WTS API can be used on a single session machine just the same as for a multi-session machine. There are several interesting functions in this API, but for this section's purposes we'll use its process enumeration functions: `WTSEnumerateProcesses` and `WTSEnumerateProcessesEx`. The WTS API is defined in its own header (not included by default with `<windows.h>`) - `<wtsapi32.h>`. It also requires adding an import library - `wtsapi32.lib` - to link successfully.

`WTSEnumerateProcesses` is defined like so:

```
typedef struct _WTS_PROCESS_INFO {
    DWORD SessionId;
    DWORD ProcessId;
    LPTSTR pProcessName;
    PSID pUserSid;
} WTS_PROCESS_INFO, *PWTS_PROCESS_INFO;
```

```
BOOL WTSEnumerateProcesses(
    _In_ HANDLE hServer,
    _In_ DWORD Reserved,
    _In_ DWORD Version,
    _Out_ PWTS_PROCESS_INFO *ppProcessInfo,
    _Out_ DWORD *pCount);
```

The function can enumerate processes on other machines with the first handle that can be obtained by calling `WTSOpenServer`. To use the local machine, the constant `WTS_CURRENT_SERVER_HANDLE` can be used instead. The `Version` parameter must be set to 1, and the real result is allocated and filled by the function itself, returning a pointer to an array of structures of type `WTS_PROCESS_INFO` for each discovered process. The last parameter returns the number of processes in the returned array. Since the function allocates the memory, the client application must free it eventually with `WTSFreeMemory`.

The following function, part of the *ProcList2* project, uses `WTSEnumerateProcesses` to show information on all processes in the system:

```
bool EnumerateProcesses1() {
    PWTS_PROCESS_INFO info;
    DWORD count;
    if (!::WTSEnumerateProcesses(WTS_CURRENT_SERVER_HANDLE, 0, 1, &info, &count\
))
        return false;

    for (DWORD i = 0; i < count; i++) {
        auto pi = info + i;
        printf("\nPID: %5d (S: %d) (User: %ws) %ws",
            pi->ProcessId, pi->SessionId,
            (PCWSTR)GetUserNameFromSid(pi->pUserSid), pi->pProcessName);
    }
    ::WTSFreeMemory(info);
}
```

```

    return true;
}

```

The returned information per-process is fairly minimal - process ID, session ID, process name (executable name or a special process name such as *System*) and a *Security Identifier* (SID) of the user running the process. The above function displays all information available, and uses a helper function to turn a SID (which is a binary blob, discussed in chapter 16) into a human-readable name:

```

CString GetUserFromSid(PSID sid) {
    if (sid == nullptr)
        return L"";

    WCHAR name[128], domain[64];
    DWORD len = _countof(name);
    DWORD domainLen = _countof(domain);
    SID_NAME_USE use;
    if (!::LookupAccountSid(nullptr, sid, name, &len, domain, &domainLen, &use))
        return L"";

    return CString(domain) + L"\\\" + name;
}

```

To link properly, `wtsapi32.lib` must be added to the linker additional dependencies in *Project/Properties*, or alternatively added in source with the appropriate `#pragma`:

```
#pragma comment(lib, "wtsapi32")
```

Running the application that calls `EnumerateProcesses1` with standard user rights shows something like the following:

```

PID:      0 (S: 0) (User: )
PID:      4 (S: 0) (User: ) System
PID:     88 (S: 0) (User: ) Secure System
PID:    152 (S: 0) (User: ) Registry
PID:    812 (S: 0) (User: ) smss.exe
PID:   1004 (S: 0) (User: ) csrss.exe
...
PID:   8904 (S: 1) (User: VOYAGER\Pavel) nvcontainer.exe
PID:   8912 (S: 1) (User: VOYAGER\Pavel) nvcontainer.exe
PID:   8992 (S: 1) (User: VOYAGER\Pavel) sihost.exe
PID:   9040 (S: 1) (User: VOYAGER\Pavel) svchost.exe
PID:   9104 (S: 0) (User: ) PresentationFontCache.exe
...

```

PID 0 (the idle process) has no name as far as the WTS APIs are concerned. As explained earlier, this is not a real process, so any name is synthetic anyway. The SID provided by the API is NULL in all non-running-user processes. To get the most out of this function, including the SIDs, we can run it in an elevated command window (or launch Visual Studio elevated and execute from the IDE). The results are better:

```

PID:      0 (S: 0) (User: )
PID:      4 (S: 0) (User: ) System
PID:     88 (S: 0) (User: NT AUTHORITY\SYSTEM) Secure System
PID:    152 (S: 0) (User: NT AUTHORITY\SYSTEM) Registry
PID:    812 (S: 0) (User: NT AUTHORITY\SYSTEM) smss.exe
PID:   1004 (S: 0) (User: NT AUTHORITY\SYSTEM) csrss.exe
...
PID:   1360 (S: 0) (User: Font Driver Host\UMFD-0) fontdrvhost.exe
PID:   1388 (S: 0) (User: NT AUTHORITY\LOCAL SERVICE) WUDFHost.exe
PID:   1492 (S: 0) (User: NT AUTHORITY\NETWORK SERVICE) svchost.exe
PID:   1540 (S: 0) (User: NT AUTHORITY\SYSTEM) svchost.exe
PID:   1608 (S: 0) (User: NT AUTHORITY\LOCAL SERVICE) WUDFHost.exe
PID:   1684 (S: 1) (User: NT AUTHORITY\SYSTEM) winlogon.exe
PID:   1760 (S: 1) (User: Font Driver Host\UMFD-1) fontdrvhost.exe
...
PID:   8396 (S: 0) (User: NT AUTHORITY\NETWORK SERVICE) svchost.exe
PID:   8904 (S: 1) (User: VOYAGER\Pavel) nvcontainer.exe
PID:   8912 (S: 1) (User: VOYAGER\Pavel) nvcontainer.exe
PID:   8992 (S: 1) (User: VOYAGER\Pavel) sihost.exe

```

```
PID: 9040 (S: 1) (User: VOYAGER\Pavel) svchost.exe
...
```

The `WTSEnumerateProcessesEx` API (available from Windows 7) is an extended version of `WTSEnumerateProcesses` that provides more information per process:

```
BOOL WTSEnumerateProcessesEx(
    _In_ HANDLE hServer,
    _Inout_ DWORD *pLevel,
    _In_ DWORD SessionID,
    _Out_ PTSTR *pProcessInfo,
    _Out_ DWORD *pCount);
```

The `pLevel` parameter can be set to 0 or 1. With 1, an array of extended structures is returned with more information for each process:

```
typedef struct _WTS_PROCESS_INFO_EX {
    DWORD SessionId;
    DWORD ProcessId;
    LPCTSTR pProcessName;
    PSID pUserSid;
    DWORD NumberOfThreads;
    DWORD HandleCount;
    DWORD PagefileUsage;
    DWORD PeakPagefileUsage;
    DWORD WorkingSetSize;
    DWORD PeakWorkingSetSize;
    LARGE_INTEGER UserTime;
    LARGE_INTEGER KernelTime;
} WTS_PROCESS_INFO_EX, *PWTS_PROCESS_INFO_EX;
```

The `SessionID` parameter allows enumerating processes in a specific session of interest, but `WTS_ANY_SESSION` can be supplied to indicate all sessions are of interest.

The `ppProcessInfo` is typed as a pointer to a pointer to a string, which makes no sense. It must be a pointer to `PWTS_PROCESS_INFO` or `PWTS_PROCESS_INFO_EX` based on the level value. The following is a revised enumeration function that uses the extended structure:

```

bool EnumerateProcesses2() {
    PWTS_PROCESS_INFO_EX info;
    DWORD count;
    DWORD level = 1;    // extended info
    if (!::WTSEnumerateProcessesEx(WTS_CURRENT_SERVER_HANDLE, &level,
        WTS_ANY_SESSION, (PWSTR*)&info, &count))
        return false;

    for (DWORD i = 0; i < count; i++) {
        auto pi = info + i;
        printf("\nPID: %5d (S: %d) (T: %3d) (H: %4d) (CPU: %ws) (U: %ws) %ws",
            pi->ProcessId, pi->SessionId, pi->NumberOfThreads, pi->HandleCount,
            (PCWSTR)GetCpuTime(pi),
            (PCWSTR)GetUserNameFromSid(pi->pUserSid), pi->pProcessName);
    }
    ::WTSFreeMemoryEx(WTSProcessInfoLevel1, info, count);

    return true;
}

```

The fields related to memory in `WTS_PROCESS_INFO_EX` (`PagefileUsage`, `PeakPagefileUsage`, `WorkingSetSize` and `PeakWorkingSetSize`) are actually buggy because they are 32 bit in size even in 64-bit processes (the structure should have been modified so that `DWORD` is changed to `DWORD_PTR` at the very least), so if one of these counters is above 4 GB, they will show wrong numbers.

Freeing the allocated memory block must be done with a dedicated function (`WTSFreeMemoryEx`), with the appropriate level related enumeration. The code displays some additional data compared to `EnumerateProcesses1` - number of threads in the process, number of handles and the total CPU time used. The `GetCpuTime` helper function returns a string representation of the CPU time:

```

CString GetCpuTime(PWTS_PROCESS_INFO_EX pi) {
    auto totalTime = pi->KernelTime.QuadPart + pi->UserTime.QuadPart;
    return CTimeSpan(totalTime / 10000000LL).Format(L"%D:%H:%M:%S");
}

```

The `KernelTime` and `UserTime` members of `WTS_PROCESS_INFO_EX` are the time the process' threads spent in kernel mode and user mode, respectively. This time is provided

in 100-nanosecond units (100 times 10 to the -9th power), which is very common within the Windows API. This means converting to seconds requires dividing by 10 million. The above code uses the `CTimeSpan` ATL helper class that accepts a number of seconds and can provide simple string formatting for the value.

Here is some sample output from running `EnumerateProcesses2` with admin rights:

```
PID:      0 (S: 0) (T: 12) (H: 0) (CPU: 7:16:22:09) (User: )
PID:      4 (S: 0) (T: 365) (H: 27610) (CPU: 0:00:14:39) (User: ) System
PID:     88 (S: 0) (T: 0) (H: 0) (CPU: 0:00:00:00) (User: NT AUTHORITY\SYSTEM) Secure System
PID:    152 (S: 0) (T: 4) (H: 0) (CPU: 0:00:00:01) (User: NT AUTHORITY\SYSTEM) Registry
PID:    812 (S: 0) (T: 2) (H: 53) (CPU: 0:00:00:00) (User: NT AUTHORITY\SYSTEM) smss.exe
PID:   1004 (S: 0) (T: 14) (H: 951) (CPU: 0:00:00:05) (User: NT AUTHORITY\SYSTEM) csrss.exe
...
```

Using the Native API

The last option available for process (and thread) enumeration is using the native API exposed by *NtDll.dll*. This API is mostly undocumented, and in some cases partially documented. Some of the documentation is part of the *Windows Driver Kit* (WDK), as some kernel APIs are the targets of native functions and so share a prototype. In user mode, Microsoft provides very partial definitions for the native API in the file `<winternl.h>`.

One of the functions is `NtQuerySystemInformation`. This mega-function can return various pieces of information depending on a `SYSTEM_INFORMATION_CLASS` that specifies the type of information requested. Here is the definition from `<winternl.h>`:

```
typedef enum _SYSTEM_INFORMATION_CLASS {
    SystemBasicInformation = 0,
    SystemPerformanceInformation = 2,
    SystemTimeOfDayInformation = 3,
    SystemProcessInformation = 5,
    SystemProcessorPerformanceInformation = 8,
    SystemInterruptInformation = 23,
    SystemExceptionInformation = 33,
    SystemRegistryQuotaInformation = 37,
    SystemLookasideInformation = 45,
    SystemCodeIntegrityInformation = 103,
    SystemPolicyInformation = 134,
} SYSTEM_INFORMATION_CLASS;
```

As can be seen from the enumeration, there are many gaps of undocumented information classes. `SystemProcessInformation` is one value that retrieves data for all processes in the system, with more fields than are available with `WTS_PROCESS_INFO_EX`. It even includes all threads for each process. In fact, *Task Manager* and *Process Explorer* use this to get process information.

Using the above enumeration returns objects of type `SYSTEM_PROCESS_INFORMATION`, which is declared in `<winternl.h>` like so:

```
typedef struct _SYSTEM_PROCESS_INFORMATION {
    ULONG NextEntryOffset;
    ULONG NumberOfThreads;
    BYTE Reserved1[48];
    UNICODE_STRING ImageName;
    KPRIORITY BasePriority;
    HANDLE UniqueProcessId;
    PVOID Reserved2;
    ULONG HandleCount;
    ULONG SessionId;
    PVOID Reserved3;
    SIZE_T PeakVirtualSize;
    SIZE_T VirtualSize;
    ULONG Reserved4;
    SIZE_T PeakWorkingSetSize;
    SIZE_T WorkingSetSize;
```

```

PVOID Reserved5;
SIZE_T QuotaPagedPoolUsage;
PVOID Reserved6;
SIZE_T QuotaNonPagedPoolUsage;
SIZE_T PagefileUsage;
SIZE_T PeakPagefileUsage;
SIZE_T PrivatePageCount;
LARGE_INTEGER Reserved7[6];
} SYSTEM_PROCESS_INFORMATION, *PSYSTEM_PROCESS_INFORMATION;

```

Although it provides quite a bit of information, there are many members named “reserved”-something, which are actually undocumented fields, rather than true reserved fields. Can we get the details for these “reserved” fields?

Although these structures and enumerations are undocumented, some have been located in some leaked Windows sources, reverse engineered or are available in the public symbols provided by Microsoft. One application that uses many of these officially undocumented functions and types is *Process Hacker*, a open-source *Process Explorer* clone available on Github. One of its sibling projects is *phnt*, which contains the largest definitions of native APIs, structures, enumerations and definitions (<https://github.com/processhacker/phnt>) that I know of.

As a quick example, the complete SYSTEM_PROCESS_INFORMATION looks something like the following:

```

typedef struct _SYSTEM_PROCESS_INFORMATION {
    ULONG NextEntryOffset;
    ULONG NumberOfThreads;
    LARGE_INTEGER WorkingSetPrivateSize;
    ULONG HardFaultCount;
    ULONG NumberOfThreadsHighWatermark;
    ULONGLONG CycleTime; // since WIN7
    LARGE_INTEGER CreateTime;
    LARGE_INTEGER UserTime;
    LARGE_INTEGER KernelTime;
    UNICODE_STRING ImageName;
    KPRIORITY BasePriority;
    HANDLE UniqueProcessId;
    HANDLE InheritedFromUniqueProcessId;
    ULONG HandleCount;

```

```

    ULONG SessionId;
    ULONG_PTR UniqueProcessKey;
    SIZE_T PeakVirtualSize;
    SIZE_T VirtualSize;
    ULONG PageFaultCount;
    SIZE_T PeakWorkingSetSize;
    SIZE_T WorkingSetSize;
    SIZE_T QuotaPeakPagedPoolUsage;
    SIZE_T QuotaPagedPoolUsage;
    SIZE_T QuotaPeakNonPagedPoolUsage;
    SIZE_T QuotaNonPagedPoolUsage;
    SIZE_T PagefileUsage;
    SIZE_T PeakPagefileUsage;
    SIZE_T PrivatePageCount;
    LARGE_INTEGER ReadOperationCount;
    LARGE_INTEGER WriteOperationCount;
    LARGE_INTEGER OtherOperationCount;
    LARGE_INTEGER ReadTransferCount;
    LARGE_INTEGER WriteTransferCount;
    LARGE_INTEGER OtherTransferCount;
    SYSTEM_THREAD_INFORMATION Threads[1];
} SYSTEM_PROCESS_INFORMATION, *PSYSTEM_PROCESS_INFORMATION;

```

Most of the fields that don't exist in the documented structures can still be obtained by other API functions. For example, process creation time can be retrieved with `GetProcessTimes`, but this does require opening a handle to the process (with access mask `PROCESS_QUERY_INFORMATION` in this case). Clearly, getting much of the information in one stroke is a win, which is why process information tools typically use the native API.

In this book, we won't use the native API unless there is a very good reason to do so. You can explore the code of *Process Hacker* to get a sense of how to use these APIs. In any case, if an official API exists, it's always safer to go with that, rather than to rely on a native undocumented API.

Exercises

1. Write a GUI or console application called *MiniProcExp* - a mini Process Explorer, that uses either the Toolhelp APIs, WTS APIs or the native API to show information about

processes. Anything that is not readily available get by opening a proper handle to the process and using the correct function to get the information. (A basic app is provided in the *MinProcExp* project).

2. Extend the previous application by adding operations on processes, such as terminating and changing priority class.
3. Continue extending the application in any way you see fit!

Summary

Processes are the most fundamental building block in Windows. It holds a set of resources such as an address space, that allows threads to execute code using these resources. In the next chapter, we'll look at how processes can be managed as a unit using *Jobs*.